

# SKIN CANCER DETECTION USING DEEP LEARNING

*Minor project-II report submitted  
in partial fulfillment of the requirement for award of the degree of*

**Bachelor of Technology  
in  
Artificial Intelligence and Machine Learning**

**By**

**B.Ashwin kumar (22UEAM0006) (VTU21349)  
I.Vishwa (22UEAM0066) (VTU22463)**

*Under the guidance of  
Dr. MADONNA ARIETH. R  
Associate Professor*



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING  
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF  
SCIENCE & TECHNOLOGY**

**(Deemed to be University Estd u/s 3 of UGC Act, 1956)**

**Accredited by NAAC with A++ Grade  
CHENNAI 600 062, TAMILNADU, INDIA**

**April, 2025**

# **SKIN CANCER DETECTION USING DEEP LEARNING**

*Minor project-II report submitted  
in partial fulfillment of the requirement for award of the degree of*

**Bachelor of Technology  
in  
Artificial Intelligence and Machine Learning**

**By**

**B.Ashwin kumar (22UEAM0006) (VTU21349)  
I.Vishwa (22UEAM0066) (VTU22463)**

*Under the guidance of  
Dr. MADONNA ARIETH. R.  
Associate Professor*



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING  
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF  
SCIENCE & TECHNOLOGY**

**(Deemed to be University Estd u/s 3 of UGC Act, 1956)  
Accredited by NAAC with A++ Grade  
CHENNAI 600 062, TAMILNADU, INDIA**

**April, 2025**

# CERTIFICATE

It is certified that the work contained in the project report titled "SKIN CANCER DETECTION USING DEEP LEARNING" by "B.Ashwin kumaar (22UEAM0006), I.Vishwa (22UEAM0066), " has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

**Signature of Supervisor**

**Dr. Madonna Arieth. R.**

**Associate Professor**

**Artificial Intelligence and Machine Learning**

**School of Computing**

**Vel Tech Rangarajan Dr. Sagunthala R&D**

**Institute of Science & Technology**

**April, 2025**

**Signature of Head of the Department**

**Dr. S. Alex David**

**Professor & Head**

**Artificial Intelligence and Machine Learning**

**School of Computing**

**Vel Tech Rangarajan Dr. Sagunthala R&D**

**Institute of Science & Technology**

**April , 2025**

**Signature of the Dean**

**Dr. S P. Chokkalingam**

**Professor & Dean**

**School of Computing**

**Vel Tech Rangarajan Dr. Sagunthala R&D**

**Institute of Science & Technology**

**April, 2025**

# DECLARATION

We declare that this written submission represents my ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)

B.Ashwin Kumar

Date:        /        /

(Signature)

I.Vishwa

Date:        /        /



# APPROVAL SHEET

This project report entitled "SKIN CANCER DETECTION USING DEEP LEARNING" by B.Ashwin Kumar (22UEAM0006), I.VISHWA (22UEAM0066) is approved for the degree of B.Tech in Artificial Intelligence and Machine Learning.

**Examiners**

**Supervisor**

Dr.MADONNA ARIETH.R

Associate professor

**Date:**        /        /

**Place:**

# ACKNOWLEDGEMENT

We express our deepest gratitude to our **Honorable Founder Chancellor and President Col. Prof. Dr. R. RANGARAJAN B.E. (Electrical), B.E. (Mechanical), M.S (Automobile), D.Sc., and Foundress President Dr. R. SAGUNTHALA RANGARAJAN M.B.B.S.** Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, for her blessings.

We express our sincere thanks to our respected Chairperson and Managing Trustee **Mrs. RANGARAJAN MAHALAKSHMI KISHORE,B.E., Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, for her blessings.**

We are very much grateful to our beloved **Vice Chancellor Prof. Dr.RAJAT GUPTA,** for providing us with an environment to complete our project successfully.

We record indebtedness to our **Professor & Dean, School of Computing, Dr. S P. CHOKKALINGAM, M.Tech., Ph.D., & Associate Dean, Dr. V. DHILIP KUMAR,M.E.,Ph.D.,** for immense care and encouragement towards us throughout the course of this project.

We are thankful to our **Professor & Head, Department of Artificial Intelligence and Machine Learning, Dr. S. ALEX DAVID, Ph.D.,** for providing immense support in all our endeavors.

We also take this opportunity to express a deep sense of gratitude to our **Internal Supervisor Supervisor name,degree.,(in capital letters)** for his/her cordial support, valuable information and guidance, he/she helped us in completing this project through various stages.

A special thanks to our **Project Coordinators Dr. B.PRABHU SHANKAR, Ph.D.,** for their valuable guidance and support throughout the course of the project.

We thank our department faculty, supporting staff and friends for their help and guidance to complete this project.

|                       |                     |
|-----------------------|---------------------|
| <b>B.Ashwin kumar</b> | <b>(22UEAM0006)</b> |
| <b>I.Vishwa</b>       | <b>(22UEAM0066)</b> |

## ABSTRACT

Skin cancer remains one of the most widespread and life-threatening cancers globally, with millions of new cases diagnosed each year. Its incidence continues to rise due to various factors including prolonged exposure to ultraviolet radiation, genetic predisposition, and environmental influences. The early detection and accurate classification of skin lesions are vital for effective treatment, significantly improving patient survival rates and outcomes. Manual diagnosis, however, remains a time-consuming and expertise-dependent process, often requiring dermatologists to analyze dermoscopic images and perform biopsies. To overcome these limitations and to provide scalable diagnostic support, this project explores the use of deep learning, particularly Convolutional Neural Networks (CNNs), for automated skin lesion classification.

This research introduces a CNN-based framework that automatically classifies skin lesions into seven distinct categories using the HAM10000 dataset. These categories include Melanoma, Basal Cell Carcinoma, Benign Keratosis, Nevus, Actinic Keratosis, Dermatofibroma, and Vascular Lesions. The dataset contains high-resolution dermoscopic images collected from multiple medical institutions and experts worldwide. To ensure uniformity and efficient computation, all images are resized to 28x28 pixels and converted into RGB format.

**Key Words:** Convolutional Neural Networks (CNN) ,Deep Learning, Medical Image Classification, HAM10000 Dataset .

# LIST OF FIGURES

|     |                                      |    |
|-----|--------------------------------------|----|
| 4.1 | Architecture Diagram . . . . .       | 10 |
| 4.2 | Data Flow Diagram . . . . .          | 12 |
| 4.3 | Use Case Diagram . . . . .           | 13 |
| 4.4 | Class Diagram . . . . .              | 15 |
| 4.5 | Sequence Diagram . . . . .           | 17 |
| 4.6 | Collaboration diagram . . . . .      | 20 |
| 4.7 | Activity Diagram . . . . .           | 21 |
| 5.1 | Input Design . . . . .               | 31 |
| 5.2 | Output Design . . . . .              | 32 |
| 5.3 | Unit Testing output . . . . .        | 35 |
| 5.4 | Integration Testing output . . . . . | 38 |
| 5.5 | System Testing output . . . . .      | 41 |
| 5.6 | Test Image 1 . . . . .               | 43 |
| 5.7 | Test Image 2 . . . . .               | 43 |
| 5.8 | Test Image 3 . . . . .               | 44 |
| 6.1 | Output 1 . . . . .                   | 52 |
| 6.2 | Output 2 . . . . .                   | 53 |

# LIST OF TABLES

|     |                                               |    |
|-----|-----------------------------------------------|----|
| 6.1 | Decision Table . . . . .                      | 49 |
| 6.2 | HAM10000 Dataset Class Distribution . . . . . | 49 |

# LIST OF ACRONYMS AND ABBREVIATIONS

|           |                                                                        |
|-----------|------------------------------------------------------------------------|
| AI        | Artificial Intelligence                                                |
| CNN       | Convolutional Neural Network                                           |
| DL        | Deep Learning                                                          |
| DT        | Decision Tree                                                          |
| HAM10000  | Human Against Machine with 10000 training images (skin cancer dataset) |
| ML        | Machine Learning                                                       |
| MaxPool2D | Max Pooling 2D (Downsampling Layer in CNN)                             |
| FLASK     | Lightweight Python Web Framework                                       |
| JPEG      | Joint Photographic Experts Group (Image Format)                        |
| BCC       | Basal Cell Carcinoma                                                   |
| AKIEC     | Actinic Keratoses and Intraepithelial Carcinomae                       |
| NV        | Melanocytic Nevi                                                       |
| MEL       | Melanoma                                                               |
| VASC      | Vascular Lesion                                                        |
| DF        | Dermatofibroma                                                         |
| BKL       | Benign Keratosis-like Lesion                                           |
| RGB       | Red Green Blue (Color Channels)                                        |
| CSV       | Comma Separated Values                                                 |
| GPU       | Graphics Processing Unit                                               |
| UI        | User Interface                                                         |
| API       | Application Programming Interface                                      |

# TABLE OF CONTENTS

|                                           | Page.No     |
|-------------------------------------------|-------------|
| <b>ABSTRACT</b>                           | <b>v</b>    |
| <b>LIST OF FIGURES</b>                    | <b>vi</b>   |
| <b>LIST OF TABLES</b>                     | <b>vii</b>  |
| <b>LIST OF ACRONYMS AND ABBREVIATIONS</b> | <b>viii</b> |
| <b>1 INTRODUCTION</b>                     | <b>1</b>    |
| 1.1 Introduction . . . . .                | 1           |
| 1.2 Aim of the project . . . . .          | 2           |
| 1.3 Project Domain . . . . .              | 2           |
| 1.4 Scope of the Project . . . . .        | 3           |
| <b>2 LITERATURE REVIEW</b>                | <b>1</b>    |
| 2.1 Overview of Related Work . . . . .    | 1           |
| 2.2 Comparative Studies . . . . .         | 1           |
| 2.3 Gap Identification . . . . .          | 2           |
| <b>3 PROJECT DESCRIPTION</b>              | <b>3</b>    |
| 3.1 Existing System . . . . .             | 3           |
| 3.2 Problem statement . . . . .           | 3           |
| 3.3 System Specification . . . . .        | 4           |
| 3.3.1 Hardware Specification . . . . .    | 4           |
| 3.3.2 Software Specification . . . . .    | 5           |
| 3.3.3 Standards and Policies . . . . .    | 7           |
| <b>4 METHODOLOGY</b>                      | <b>9</b>    |
| 4.1 Proposed System . . . . .             | 9           |
| 4.2 General Architecture . . . . .        | 10          |
| 4.3 Design Phase . . . . .                | 12          |
| 4.3.1 Data Flow Diagram . . . . .         | 12          |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| 4.3.2    | Use Case Diagram . . . . .                                 | 13        |
| 4.3.3    | Class Diagram . . . . .                                    | 15        |
| 4.3.4    | Sequence Diagram . . . . .                                 | 17        |
| 4.3.5    | Collaboration diagram . . . . .                            | 18        |
| 4.3.6    | Activity Diagram . . . . .                                 | 21        |
| 4.4      | Algorithm & Pseudo Code . . . . .                          | 22        |
| 4.4.1    | Algorithm . . . . .                                        | 22        |
| 4.4.2    | Pseudo Code . . . . .                                      | 23        |
| 4.4.3    | Data Set / Generation of Data (Description only) . . . . . | 24        |
| 4.5      | Module Description . . . . .                               | 26        |
| 4.5.1    | Data Preprocessing . . . . .                               | 26        |
| 4.5.2    | CNN Model . . . . .                                        | 26        |
| 4.5.3    | Machine Learning Models . . . . .                          | 27        |
| 4.5.4    | Evaluation Module . . . . .                                | 28        |
| <b>5</b> | <b>IMPLEMENTATION AND TESTING</b>                          | <b>30</b> |
| 5.1      | Input and Output . . . . .                                 | 30        |
| 5.1.1    | Input Design . . . . .                                     | 30        |
| 5.1.2    | Output Design . . . . .                                    | 31        |
| 5.2      | Testing . . . . .                                          | 32        |
| 5.3      | Types of Testing . . . . .                                 | 32        |
| 5.3.1    | Unit testing . . . . .                                     | 32        |
| 5.3.2    | Integration testing . . . . .                              | 35        |
| 5.3.3    | System testing . . . . .                                   | 38        |
| 5.3.4    | Test Result . . . . .                                      | 42        |
| <b>6</b> | <b>RESULTS AND DISCUSSIONS</b>                             | <b>45</b> |
| 6.1      | Efficiency of the Proposed System . . . . .                | 45        |
| 6.2      | Comparison of Existing and Proposed System . . . . .       | 47        |
| <b>7</b> | <b>CONCLUSION AND FUTURE ENHANCEMENTS</b>                  | <b>54</b> |
| 7.1      | Conclusion . . . . .                                       | 54        |
| 7.2      | Future Enhancements . . . . .                              | 55        |
| <b>8</b> | <b>PLAGIARISM REPORT</b>                                   | <b>56</b> |



|                                      |           |
|--------------------------------------|-----------|
| <b>Appendices</b>                    | <b>57</b> |
| <b>A Complete Data / Sample Data</b> | <b>58</b> |
| <b>B Sample Source Code</b>          | <b>60</b> |
| <b>References</b>                    | <b>66</b> |

# Chapter 1

## INTRODUCTION

### 1.1 Introduction

Skin cancer is one of the most frequently diagnosed forms of cancer worldwide, with millions of new cases reported annually. Its early and accurate detection is critical to improving treatment outcomes and patient survival rates. However, traditional diagnostic procedures, such as visual inspections followed by biopsies, are not only time-consuming but also susceptible to human error, particularly in regions with limited access to experienced dermatologists or advanced diagnostic tools. This presents a significant challenge for healthcare systems in both developed and developing countries.

With the rapid advancement of Artificial Intelligence (AI) and its sub-fields—Machine Learning (ML) and Deep Learning (DL)—medical image analysis has undergone a transformative shift. In particular, Convolutional Neural Networks (CNNs) have demonstrated remarkable success in image classification tasks due to their ability to automatically extract and learn hierarchical features from raw input data. This capability makes CNNs a powerful tool for dermatological applications, especially in detecting various types of skin lesions through dermoscopic images.

This project introduces a CNN-based intelligent system designed for the automatic classification of skin lesions. The model is trained on the publicly available HAM10000 dataset, which includes over 10,000 dermoscopic images categorized into seven classes: melanoma, basal cell carcinoma, benign keratosis, nevus, actinic keratosis, dermatofibroma, and vascular lesions. Each image is preprocessed by resizing to 28x28 pixels and normalized for optimal training performance.

To enhance usability and accessibility, the trained model is deployed via a Flask web interface that allows users to upload images and receive instant classification results. The platform is intended not to replace professional medical diagnosis but to serve as a supportive tool for early detection and public awareness. Overall, this system aims to offer a scalable, low-cost, and efficient diagnostic aid, especially in resource-constrained environments.

## 1.2 Aim of the project

The aim of this project is to develop and compare two types of models for skin cancer detection from dermoscopic images:

- **Deep Learning Approach :** Implementing a CNN model to automatically learn patterns from raw image data, facilitating high-accuracy classification.
- **Traditional Machine Learning Approaches:** Implementing machine learning models, including Random Forest, Support Vector Machine (SVM), and Decision Trees, using handcrafted features derived from the images.
- **Through this comparison, the project seeks to determine:**

Whether deep learning & CNN significantly outperforms traditional machine learning models in terms of classification accuracy and reliability. If traditional models can offer competitive results with improved interpretability and lower computational requirements, making them potentially more suitable for environments with limited resources.

## 1.3 Project Domain

- This project lies at the intersection of Medical Image Analysis and Artificial Intelligence (AI). In recent years, medical image analysis has benefited immensely from AI advancements, particularly in the area of automated diagnostics. This study contributes to this domain by focusing on image classification for skin cancer detection. By utilizing both CNNs and traditional machine learning models, the project addresses two important sub-domains:
- **Deep Learning for Image Classification:** Employing CNNs to capture complex spatial hierarchies and subtle patterns within dermoscopic images.
- **Traditional Machine Learning for Interpretability and Efficiency:** Applying traditional machine learning techniques, known for their interpretability, to assess how effectively they can classify images based on predefined, interpretable features.

## 1.4 Scope of the Project

**Dataset Selection and Preprocessing:** The HAM10000 dataset, a standard dataset for skin lesion classification, is used for training and evaluation. Preprocessing steps involve resizing, normalizing, and augmenting the images to create a robust and standardized input for both deep learning and traditional models.

### **Model Development:**

- **Convolutional Neural Network (CNN):** The project will design a CNN architecture tailored for image classification, focusing on achieving high accuracy by leveraging the spatial and hierarchical relationships in image data.
- **Traditional Machine Learning Models:** Feature extraction techniques will be applied to generate descriptive features (e.g., color histogram, texture, and shape) from images. These features will then be used to train traditional models, including Random Forest, SVM, and Decision Tree.

**Evaluation and Metrics:** The models will be evaluated on test data based on metrics such as accuracy, precision, recall, and ROC-AUC. These metrics allow for a comprehensive comparison between the approaches, considering not only the accuracy but also the models' ability to minimize false negatives, a critical factor in cancer detection.

**Comparison of Performance and Interpretability:** Beyond accuracy, interpretability is a crucial aspect of this study. CNNs, while accurate, can be "black-box" models where the decision-making process is not easily interpretable. In contrast, traditional models provide clearer decision boundaries and rules, which may make them more suitable for practical applications where model transparency is essential.

## Chapter 2

# LITERATURE REVIEW

### 2.1 Overview of Related Work

Recent studies have shown that deep learning, especially Convolutional Neural Networks (CNNs), can effectively detect skin cancer from images. Esteva et al. (2017) demonstrated that CNNs can match dermatologist-level accuracy. The HAM10000 dataset, widely used in research, has enabled models like ResNet, VGGNet, and Inception to be trained for classifying multiple skin lesion types.

Techniques such as data augmentation, transfer learning, and oversampling have been used to improve accuracy and deal with challenges like class imbalance and image quality variability. Our project builds on this by proposing a custom CNN with data balancing and a user-friendly web interface for real-time skin lesion classification.

### 2.2 Comparative Studies

Various machine learning and deep learning models have been explored for skin cancer detection. Traditional models like SVMs and Random Forests rely heavily on manual feature extraction and perform well on smaller datasets. However, they often lack scalability and accuracy when compared to deep learning approaches.

In contrast, Convolutional Neural Networks (CNNs) automatically learn spatial features from raw image data, achieving higher accuracy. For instance:

- Esteva et al. (2017): Used a deep CNN and achieved dermatologist-level classification accuracy.
- ResNet and Inception architectures: Known for their depth and residual connections, providing better performance on large-scale datasets like HAM10000.
- Our proposed custom CNN: Designed with multiple convolutional and pooling layers, BatchNormalization, and Dropout, balances accuracy and computational efficiency while handling class imbalance using RandomOverSampler.

This comparative analysis emphasizes CNNs as the most effective approach for robust and scalable skin cancer classification.

## 2.3 Gap Identification

Despite these advancements, there are gaps in the existing research:

- **Lack of Direct Comparative Studies:** While both CNNs and traditional machine learning models are used for skin cancer detection, few studies directly compare these two approaches under similar experimental conditions using a shared dataset.
- **Interpretability Challenges in Deep Learning:** CNNs are often criticized for their lack of interpretability, making it difficult for practitioners to understand the decision-making process. A comparative study focusing on both accuracy and interpretability could provide practical insights for medical professionals.
- **Resource and Deployment Considerations:** Most research emphasizes the accuracy of models but overlooks the practicality of deploying these models in diverse environments, especially where computational power may be limited.

This project aims to address these gaps by implementing and comparing CNNs with traditional models under a unified framework and evaluating each model's interpretability and practical applicability.

## Chapter 3

# PROJECT DESCRIPTION

### 3.1 Existing System

Currently, skin cancer detection relies primarily on manual examination by dermatologists using dermatoscopes. Dermatologists examine specific visual patterns and attributes within lesions, which requires extensive expertise. In clinical settings, automated diagnostic systems are sometimes available, often using either traditional machine learning algorithms with handcrafted features or deep learning models trained on large datasets.

Despite the advances, these existing systems have limitations:

- **High Dependence on Expertise:** Traditional methods require extensive feature engineering based on expert knowledge, which limits their adaptability.
- **Lack of Comprehensive Comparative Analysis:** Few clinical tools offer both deep learning and traditional machine learning models for comparison.
- **Resource Constraints:** The use of CNNs is often computationally demanding, making it difficult for resource-constrained settings to benefit from high-performance models.

### 3.2 Problem statement

The accurate and timely diagnosis of skin cancer remains a significant challenge in clinical dermatology. Despite advances in dermoscopic imaging, early detection continues to be hindered by inter-observer variability, limited access to expert dermatologists, and the subtle visual similarities between malignant and benign skin lesions.

Machine learning, particularly Convolutional Neural Networks (CNNs), offers potential to augment diagnostic capabilities through automated image classification. However, current implementations face several limitations:

- **Data Imbalance:** Most publicly available datasets, such as HAM10000, exhibit a skewed distribution of lesion classes, leading to biased model predictions.
- **High Intra-class Variability:** Benign lesions often share morphological features with malignant ones, complicating model generalization.
- **Computational Constraints:** High-performance models may be computationally expensive, limiting their deployment in low-resource settings.
- **Lack of Explainability:** Many CNN-based solutions function as black-box models, reducing their clinical trust and interpretability.

The primary objective of this research is to develop a robust CNN-based classification system capable of accurately detecting and categorizing skin lesions from dermoscopic images. The proposed model integrates data preprocessing, oversampling techniques to mitigate class imbalance, and a lightweight CNN architecture tailored for real-time application. The goal is to achieve high classification performance while ensuring computational efficiency and clinical applicability.

### **3.3 System Specification**

#### **3.3.1 Hardware Specification**

For developing and testing machine learning and deep learning models for skin cancer detection, a laptop with adequate processing power, memory, and storage is essential. Since this project involves training Convolutional Neural Networks (CNNs) and other machine learning models on image data, the laptop's hardware should meet certain minimum requirements to handle computationally intensive tasks efficiently.

#### **Minimum Recommended Hardware Specifications:**

##### **1.Processor (CPU):**

- Intel Core i5 (10th generation or higher) or AMD Ryzen 5 (3000 series or higher) with at least 4 cores and 8 threads.
- For more efficient processing, an Intel Core i7 or AMD Ryzen 7 processor is preferable, as it can handle larger datasets and more complex model architectures better.

##### **2.Graphics Card (GPU):**



- A dedicated GPU can significantly speed up the training of CNN models. NVIDIA GPUs are widely supported for deep learning, especially those with CUDA cores.
- Minimum: NVIDIA GTX 1650 or GTX 1660 with 4 GB VRAM.
- Recommended: NVIDIA RTX 2060 or higher with 6+ GB VRAM for faster training.

### **3.Memory (RAM):**

- Minimum: 8 GB RAM
- Recommended: 16 GB RAM or higher to accommodate larger datasets and reduce processing lag during model training and testing.

### **4.Storage:**

- SSD (Solid State Drive) with at least 256 GB for faster read/write speeds, as model training involves handling a large volume of data.
- Recommended: 512 GB SSD or a combination of SSD and HDD (e.g., 256 GB SSD + 1 TB HDD) to provide adequate storage for datasets and trained models.

### **5.Display:**

- A Full HD (1920 x 1080) resolution display with good color accuracy is recommended to observe image-based results and visualizations more clearly.
- Operating System: Windows 10 or 11 (64-bit) or a Linux-based OS (e.g., Ubuntu) to ensure compatibility with Python libraries and machine learning frameworks.

## **3.3.2 Software Specification**

### **1.Operating System:**

- Windows 10/11 (64-bit), macOS (for Mac users), or a Linux Distribution like Ubuntu (preferred for compatibility with many machine learning tools and libraries).

### **2.Programming Language:**

- Python 3.8 or higher: Python is essential for machine learning and deep learning projects due to its extensive support for data science libraries and tools.

### **3.Integrated Development Environment (IDE):**

- Jupyter Notebook (often part of the Anaconda package) for interactive coding and visualization.
- PyCharm (Community Edition) or Visual Studio Code for a more comprehensive coding environment.
- Google Colab (online option) for running notebooks on Google's cloud-based GPUs, which is particularly helpful if a laptop has limited processing power.

### **4.Machine Learning and Deep Learning Libraries:**

- TensorFlow (version 2.x) or PyTorch: Required for implementing and training deep learning models like CNNs.
- Keras: A high-level neural networks API, often used with TensorFlow for building and training deep learning models more intuitively.
- scikit-learn: Essential for implementing traditional machine learning algorithms (e.g., SVM, Decision Trees, Random Forest) and evaluation metrics.

### **5.Data Processing and Analysis Libraries:**

- NumPy: For numerical operations on data arrays, which are crucial for handling image data.
- Pandas: For data manipulation and preprocessing, particularly useful for structuring datasets and managing labels.
- OpenCV or Pillow (PIL): For image processing tasks, such as resizing, cropping, and augmenting images.

### **6.Visualization Tools:**

- Matplotlib and Seaborn: For creating visualizations of data distributions, training curves, and evaluation metrics.
- TensorBoard: A visualization toolkit that comes with TensorFlow, useful for tracking and visualizing model performance during training.

### **7.Version Control System:**

- Git: To manage code versions and facilitate collaboration (optional but highly recommended).

- GitHub or GitLab for hosting the project repository and sharing it with team members or instructors.

## **8.Additional Tools:**

- Anaconda: A package manager that simplifies the installation of Python, Jupyter Notebook, and other necessary libraries.
- Docker (optional): For creating and deploying containerized applications, which can help in replicating the environment for testing or deployment on different machines.

### **3.3.3 Standards and Policies**

When developing machine learning models for medical image analysis, it is essential to adhere to various standards and policies to ensure accuracy, ethical considerations, and compliance with data protection laws. This section outlines the key standards and policies relevant to this project, which aims to classify skin lesions as benign or malignant using both deep learning and traditional machine learning techniques.

#### **1. Data Privacy and Security Standards:**

**Health Insurance Portability and Accountability Act (HIPAA) (U.S.) and General Data Protection Regulation (GDPR) (EU):**

- These regulations ensure that any patient data used in machine learning research is handled securely and anonymized where necessary to protect individual privacy.
- Although the HAM10000 dataset used in this project is anonymized, all handling and usage of data must follow data protection practices, especially if applied in real-world clinical settings.

#### **Ethical Use of Data:**

- Ensure data is used exclusively for research purposes, with no sharing of personally identifiable information.
- Anonymized datasets (like HAM10000) are used to safeguard patient identities, and this approach is maintained across the entire data pipeline.

#### **2. Software Development Standards:**

**IEEE Standard for Software Quality Assurance Processes (IEEE 730):**

- This standard provides guidelines for ensuring the software is robust, efficient, and reliable, particularly in fields such as healthcare where accuracy is paramount.
- This project should implement quality assurance checks throughout the development process, including unit testing, integration testing, and system testing.

### **PEP 8 (Python Enhancement Proposal 8):**

- Adherence to Python’s coding standard, PEP 8, ensures readability and maintainability, which is essential for collaboration and future development of the project.

## **3. Model Evaluation and Performance Standards:**

### **ISO/IEC 25010:2011 (System and Software Quality Model):**

- This standard offers a framework for evaluating the quality of machine learning models based on metrics such as accuracy, precision, recall, and interpretability.
- In this project, evaluation metrics are chosen according to ISO/IEC guidelines to ensure model performance is accurately assessed.

### **Fairness, Accountability, and Transparency (FAcCT):**

- Fairness in AI models is critical, especially in healthcare applications. This project follows FAcCT principles to reduce potential biases in model training and evaluation, ensuring consistent performance across diverse populations.

## **4. Ethical AI and Responsible AI Use: IEEE Global Initiative on Ethics of Autonomous and Intelligent Systems:**

- This initiative emphasizes the ethical design and deployment of AI systems, which is particularly relevant in healthcare applications.
- For this project, considerations include ensuring the model’s predictions are interpretable and that clinicians can understand and trust the model outputs.

### **Explainability and Interpretability:**

- It is essential that models not only provide accurate results but also explain their predictions in an understandable way. Traditional machine learning models like Random Forests and Decision Trees provide interpretability, and CNN models are made more interpretable through techniques such as Grad-CAM.

# Chapter 4

## METHODOLOGY

### 4.1 Proposed System

#### Overview:

- The proposed system utilizes the Random Forest algorithm to classify skin lesions as benign or malignant, aiming for improved accuracy and robustness over traditional models. Random Forest, being an ensemble method, combines multiple decision trees to make more reliable predictions, reducing the risk of overfitting and enhancing generalization.

#### Advantages of Random Forest:

- Higher Accuracy: By averaging the predictions from multiple trees, Random Forest achieves higher accuracy in classifying skin lesions compared to a single decision tree.
- Reduced Overfitting: The ensemble nature of Random Forest mitigates the risk of overfitting, making it more robust for medical image classification.
- Feature Importance: Random Forest provides insights into feature importance, helping to identify key characteristics of skin lesions that contribute most to the classification.

#### Process:

- The algorithm is trained on the HAM10000 dataset, which contains labeled images of various skin lesion types.
- Each decision tree within the Random Forest is built on a random subset of data, ensuring diversity among trees and enhancing predictive stability.
- The final classification (benign or malignant) is determined by aggregating the predictions from all trees, leading to a more accurate overall result.

#### Evaluation Metrics:

- The system's performance is evaluated based on metrics such as accuracy, precision, recall, and F1 score, focusing on reliable classification to support early diagnosis in a clinical setting.

## 4.2 General Architecture

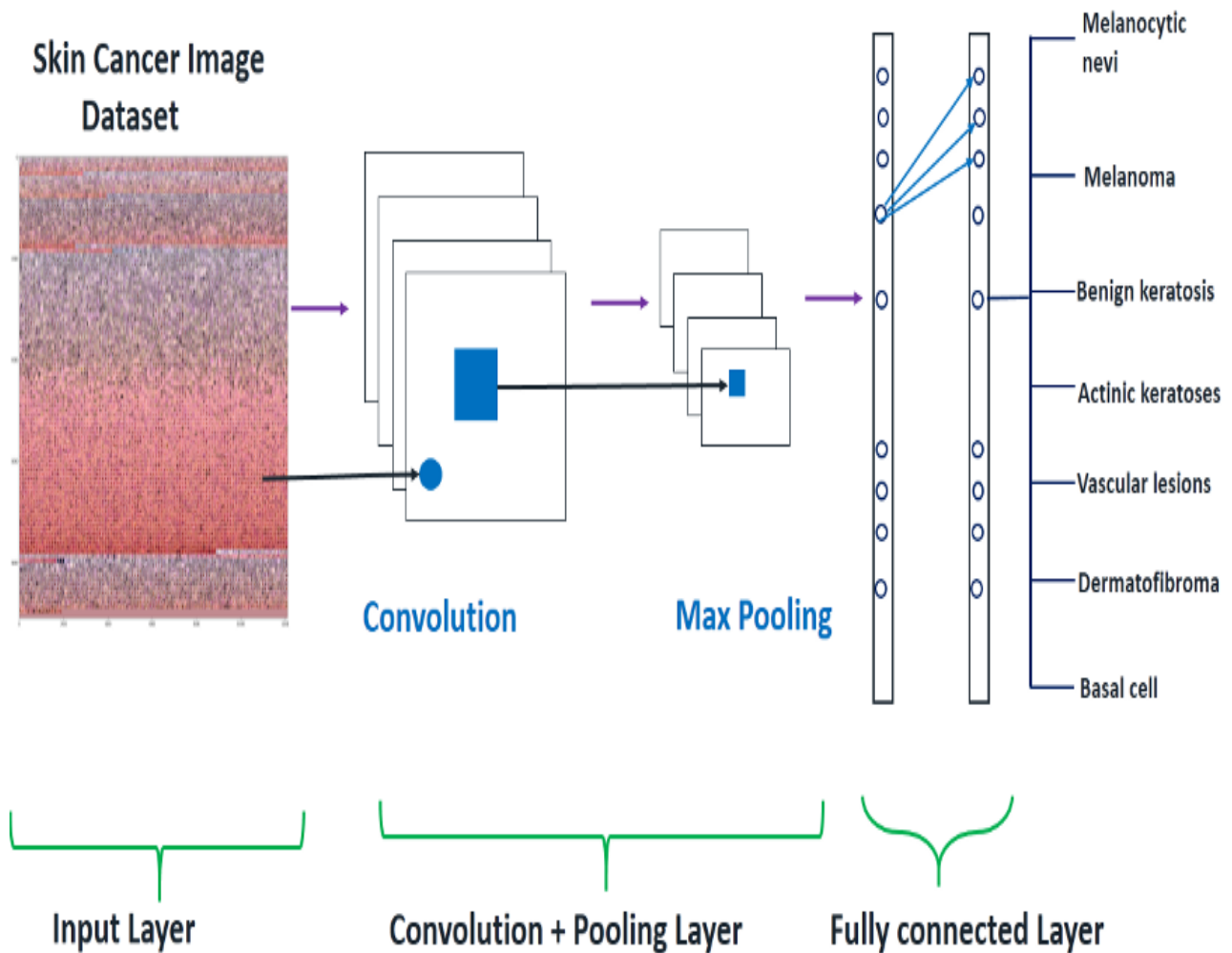


Figure 4.1: **Architecture Diagram**

In figure 4.1 the diagram is a Architecture for skin cancer detection using image classification. It outlines the process from data collection and cleaning, through segmentation and feature extraction, to model training and performance evaluation, ending with predictions of skin cancer.

### **The architecture typically consists of several stages:**

**The Data Collection and Preprocessing:** Collect and preprocess the HAM10000 dataset, which involves image resizing, normalization, and augmentation.

**Feature Extraction (CNN & DL):**

- For CNN, feature extraction is part of the deep learning process.
- For machine learning models, handcrafted feature extraction is done.

**Model Training:** Use CNNs for deep learning and train machine learning models (Random Forest, SVM, Decision Trees).

**Model Evaluation:** Assess model performance using metrics like accuracy, precision, recall, and F1-score.

**Prediction and Classification:** Predict whether an image is benign or malignant based on trained models.

**Interpretability and Visualization:** Use techniques like Grad-CAM for CNNs to interpret results.

## 4.3 Design Phase

### 4.3.1 Data Flow Diagram

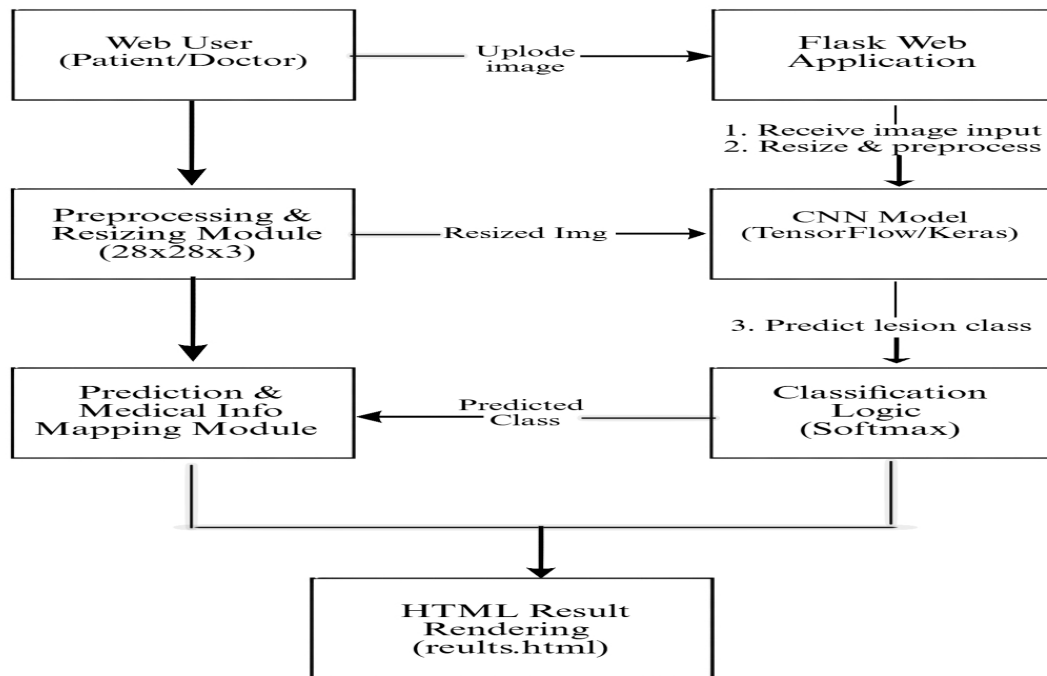


Figure 4.2: **Data Flow Diagram**

In figure 4.2 The diagram is a dataflow for skin cancer detection using image classification. It outlines the steps from data collection and cleaning, through segmentation and feature extraction, to model training and performance evaluation, ending with predictions of skin cancer.

The Data Flow Diagram (DFD) for a skin cancer detection system using CNN and machine learning can be structured to depict the flow of data through different processes in the system. Here's an outline of the DFD stages:

#### **Web User**

- Uploads the dermoscopic image through a Flask-based web interface.

#### **Flask Web Application**

- Acts as the central controller.
- Routes requests, handles preprocessing, and triggers prediction.

#### **Preprocessing Module**

- Resizes the image to 28x28.



- Converts it into a NumPy array.
- Normalizes and reshapes it to fit the CNN input layer.

### **CNN Model**

- A custom-built Convolutional Neural Network.
- Predicts one of the 7 skin disease classes using a Softmax activation function.

### **Prediction Mapping**

- Maps predicted index to its corresponding label from the classes dictionary.
- Fetches associated medical info.

### **Output Rendering**

- Displays prediction and description in results.html.

#### **4.3.2 Use Case Diagram**

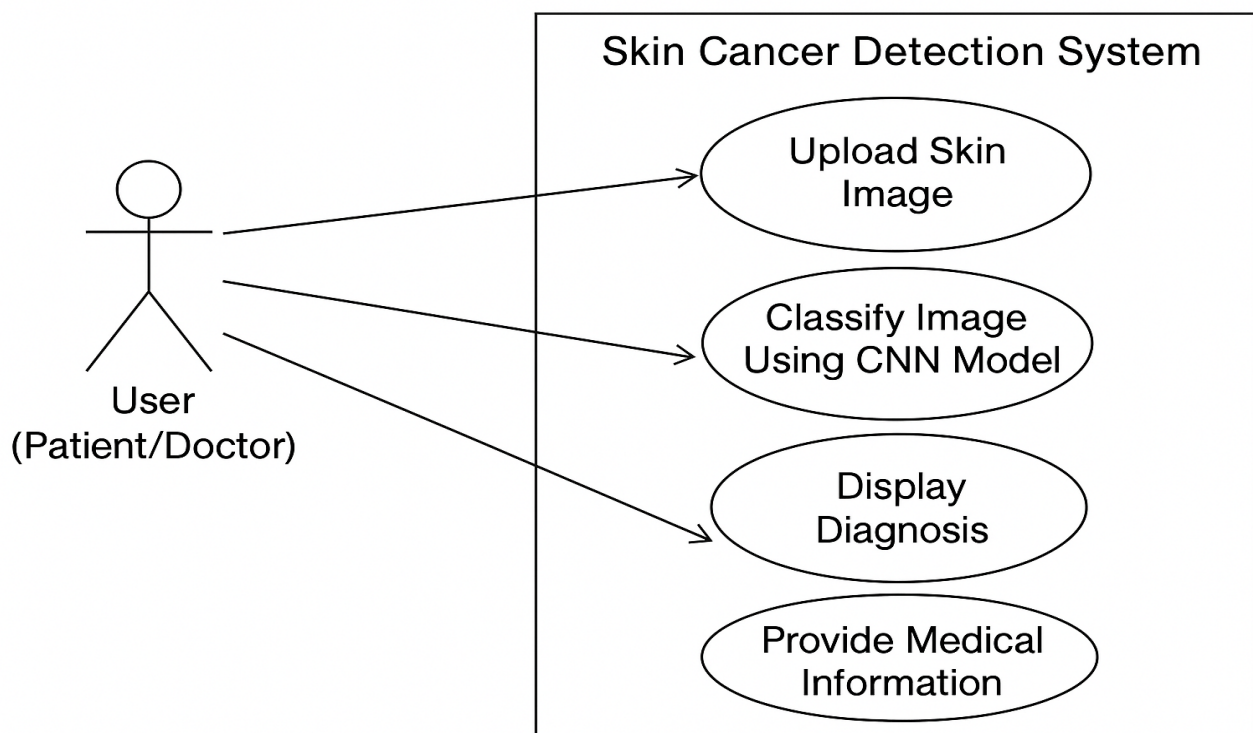


Figure 4.3: Use Case Diagram

In figure 4.3 A use case diagram for skin cancer detection using deep learning and CNN would illustrate the interactions between users (actors) and the system.

Actors :

- User: Typically, this might be a clinician, researcher, or other end-user analyzing skin images.
- System: The automated system handling image processing, classification, and displaying results.

### **Use Cases:**

- Upload Image: User uploads a dermatology image for analysis.
- Preprocess Image: The system preprocesses the image (resizing, normalization, augmentation).
- Select Model Type: User chooses between deep learning (CNN) or traditional machine learning (Random Forest, SVM, Decision Tree).
- Train Model: System trains the model on the input data, if training is required.
- Classify Image: System classifies the image as benign or malignant.
- Evaluate Model Performance: The system calculates metrics (accuracy, precision, recall) for each classification.
- Generate Interpretability Visuals: System generates interpretability visuals, such as heatmaps using Grad-CAM.
- View Results: User views the classification result and interpretability visuals.

### 4.3.3 Class Diagram

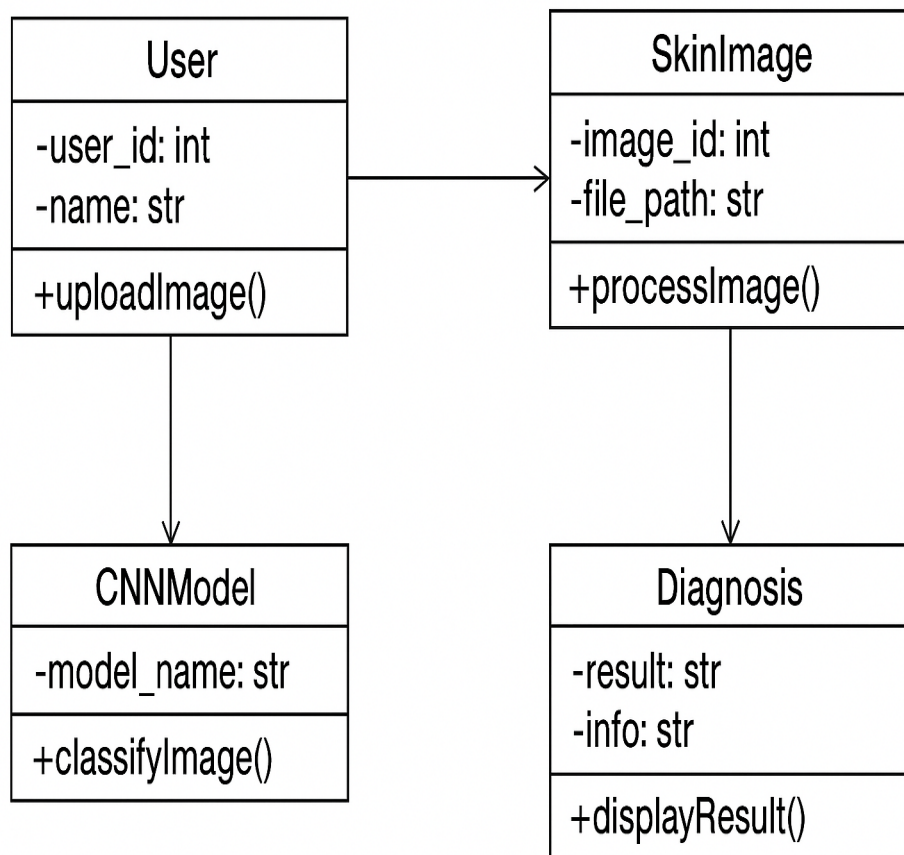


Figure 4.4: **Class Diagram**

In figure 4.4 A class diagram for a skin cancer detection system using deep learning and CNN would illustrate the system's structure by showing its classes, attributes, methods, and relationships.

Shows the Classes and Attributes:

#### **ImageProcessor**

- Attributes: image\_data, preprocessed\_data
- Methods: resize(), normalize(), augment()

#### **FeatureExtractor**

- Attributes: features
- Methods: extract\_features()

#### **Model (abstract class)**

- Attributes: accuracy, precision, recall

- Methods: train(), predict(), evaluate()

### **CNNModel (inherits from Model)**

- Attributes: layers, filters, kernel\_size
- Methods: forward\_propagate()

### **MachineLearningModel (inherits from Model)**

- Attributes: algorithm (e.g., Random Forest, SVM, Decision Tree)
- Methods: train\_model(), predict\_with\_ml()

### **Evaluation**

- Attributes: true\_labels, predictions
- Methods: calculate\_accuracy(), calculate\_precision(), calculate\_recall()

### **InterpretabilityModule**

- Attributes: grad\_cam\_visualization
- Methods: generate\_heatmap()

### **UserInterface**

- Attributes: results\_display, interpretability\_display
- Methods: display\_results(), display\_visualization()

### **Relationships:**

- UserInterface uses Model and Interpretability Module to display results and visuals.
- ImageProcessor provides processed images to both CNNModel and Machine-LearningModel.
- has an association with Evaluation for performance assessment.

#### 4.3.4 Sequence Diagram

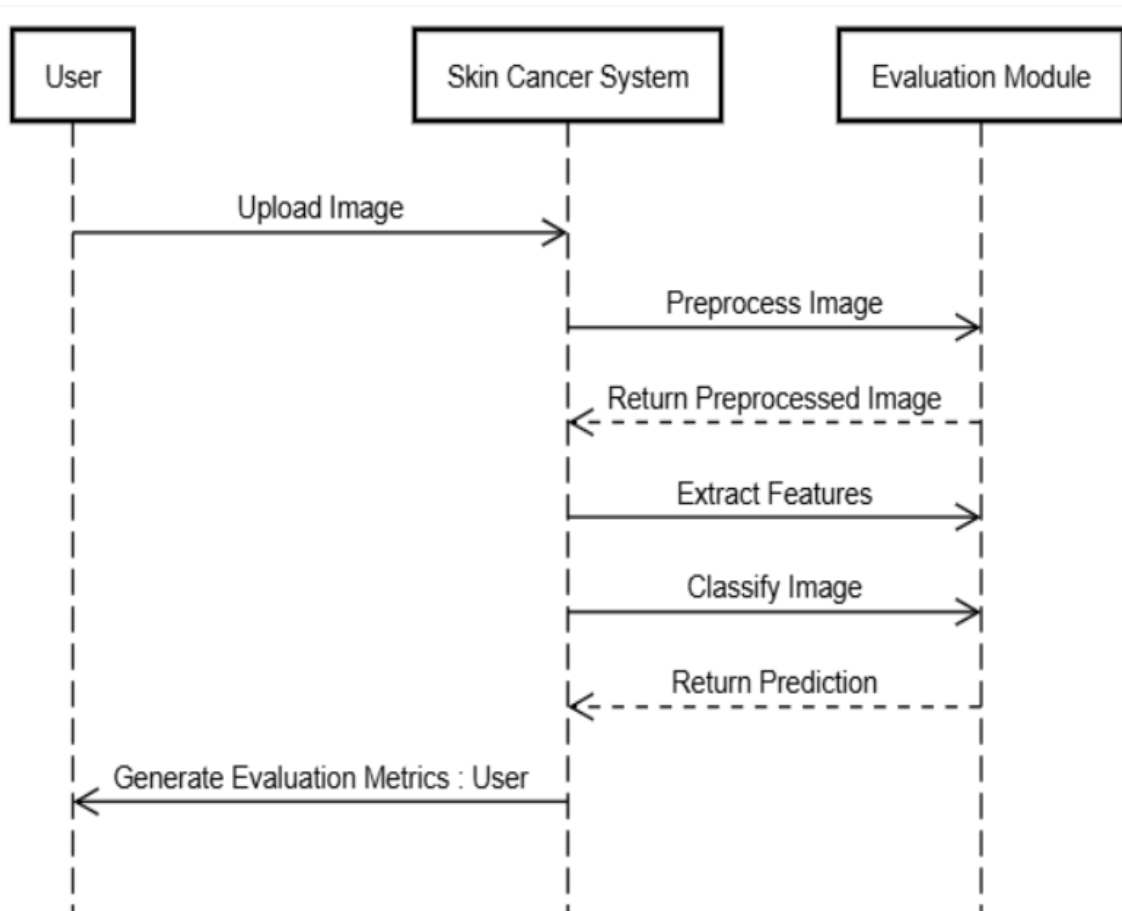


Figure 4.5: **Sequence Diagram**

In figure 4.5 A Sequence diagram for skin cancer detection using deep learning and CNN shows the system's structure, including classes like Patient, Image, Diagnosis, Model, and Doctor, along with their attributes and relationships.

A Sequence Diagram for this skin cancer detection project can illustrate the order of operations from image input to prediction and visualization output. Here's an outline of how each step in the process might look:

##### **Sequence Diagram Outline**

##### **1. Actors/Objects:**

- User
- UserInterface
- ImageProcessor
- Model (CNNModel or MachineLearningModel)
- Evaluation

- InterpretabilityModule

## **2.Sequence of Interactions:**

- User initiates Image Upload through UserInterface.
- UserInterface sends the uploaded image to ImageProcessor.
- ImageProcessor performs preprocessing (resize, normalize, augment) and returns processed image data to UserInterface.
- UserInterface forwards the preprocessed image to the Model.

## **3.Model (either CNN Model or Machine Learning Model) processes the image:**

- If it's CNN Model, it applies convolutional layers and predicts whether the lesion is benign or malignant.
- If it's MachineLearningModel, it uses the extracted features to classify the lesion.
- Model sends the prediction results to Evaluation.
- Evaluation calculates metrics (accuracy, precision, recall) and sends these to UserInterface.
- UserInterface requests Interpretability Module to generate a heatmap or Grad-CAM visualization for the prediction.
- Interpretability Module processes the request and returns the visualization to UserInterface.
- UserInterface displays the classification result, performance metrics, and interpretability visualization to the User.

### **4.3.5 Collaboration diagram**

#### **Components of the Collaboration Diagram Actors:**

- User (e.g., Doctor, Radiologist)
- System (Skin Cancer Detection System)

#### **Objects:**

- Image Preprocessing Module
- CNN Model
- ML Models (Random Forest, SVM, Decision Trees)
- Evaluation Module

**Messages:**

- Upload Image
- Boundary Localization
- Extract Features
- Normalization
- Input Image
- CNN
- Deep Learning
- classifier
- Return Prediction
- Generate Evaluation Metrics

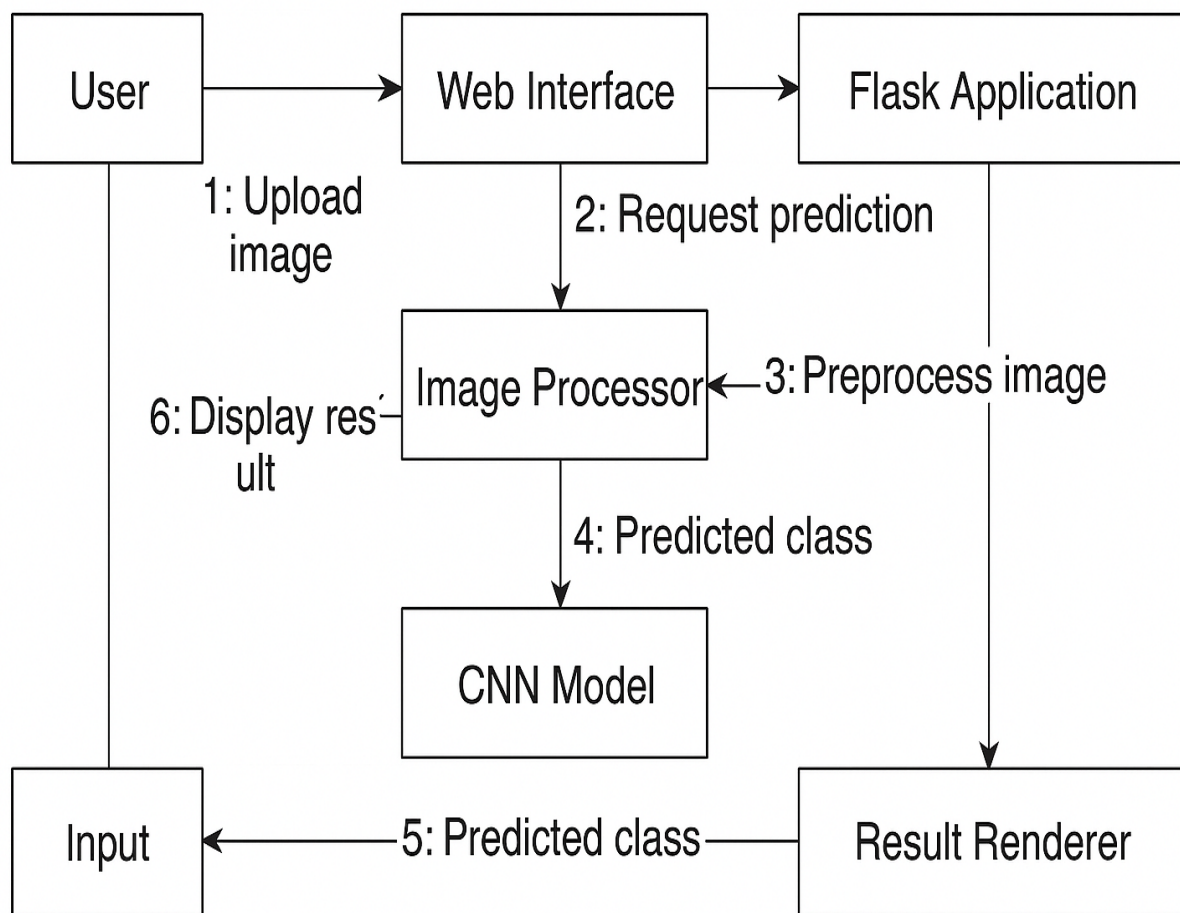


Figure 4.6: **Collaboration diagram**

In figure 4.6 A collaboration diagram for skin cancer detection using deep learning and CNN shows the interactions between objects in the system.

Actors and Objects for Collaboration Diagram :

- **Identify the Actors:** Determine who will interact with the system (e.g., users, systems).
- **Define the Objects** Identify the components of your system that will interact.
- **Establish Relationships:** Show how these objects are related and the messages passed between them.



- **Use Notations:** Use arrows to indicate the direction of the message flow and include sequence numbers if necessary.

#### 4.3.6 Activity Diagram

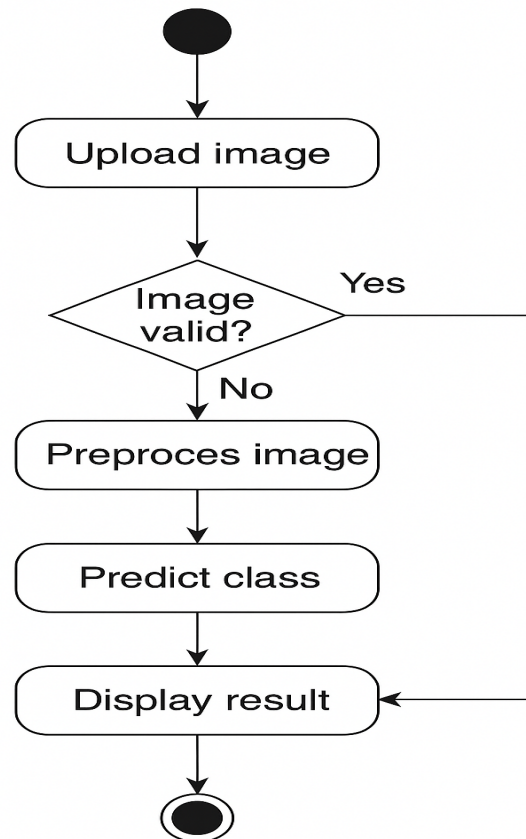


Figure 4.7: **Activity Diagram**

In figure 4.7 An activity diagram for skin cancer detection using deep learning and CNN illustrates the workflow of the system.

#### The Tools to Create Diagrams

You can use various tools to create visual collaboration diagrams, such as:

- Capture Image
- Preprocess Image
- Detect Skin cancer
- Detection
- Generate Report

- End Detection
- End Skin Cancer Detection

## 4.4 Algorithm & Pseudo Code

### 4.4.1 Algorithm

The algorithm here will describe the high-level steps for building a skin cancer detection model using a CNN and comparing it with traditional machine learning models. The overall algorithm involves data preprocessing, model building, training, and evaluation.

#### **Algorithm for Skin Cancer Detection Model Data Collection:**

Load the HAM10000 dataset, containing labeled skin lesion images.

#### **Data Preprocessing:**

- Resize images to a standard size (e.g., 128x128).
- Normalize pixel values to scale between 0 and 1.
- Apply data augmentation techniques (e.g., rotation, flipping) to enhance dataset variability.
- **Split the Dataset:** Divide the dataset into training, validation, and testing subsets.
- **Build Models: CNN Model:** Design a CNN architecture with multiple convolutional and pooling layers.
- **ML Models:** Set up models such as Random Forest, SVM, and Decision Tree using features extracted from the preprocessed images.
- **Feature Extraction for ML Models:** Use a CNN-based feature extractor or another method (e.g., SIFT or HOG) to obtain image features for input into ML models.
- **Train Models:** Train the CNN on the training dataset, optimizing with techniques like early stopping and dropout.
- Train ML models using extracted features from the training set.
- **Evaluate Models:**

- Test each model on the testing dataset.
- Calculate accuracy, precision, recall, F1 score, and ROC-AUC for each model.
- **Compare Results:** Compare the CNN and ML models based on their evaluation metrics.
- **Output Prediction:** For a given input image, provide a prediction on whether the lesion is benign or malignant.

#### 4.4.2 Pseudo Code

```

1 BEGIN
2   LOAD HAM10000 dataset
3
4   FOR each image in dataset
5     RESIZE image to 128x128
6     NORMALIZE pixel values
7     PERFORM data augmentation (if training image)
8   END FOR
9
10  SPLIT dataset into training , validation , and testing sets
11
12  // Build and train CNN model
13  INITIALIZE CNN model with layers (Conv -> Pool -> Dense)
14  COMPILE model with loss function and optimizer
15  FIT model on training data with validation on validation set
16  SAVE best model based on validation performance
17
18  // Train ML models with feature extraction
19  FOR each image in training set
20    EXTRACT features using CNN or feature extraction method
21  END FOR
22
23  INITIALIZE ML models (Random Forest , SVM, Decision Tree)
24  TRAIN each ML model on extracted features
25
26  // Evaluate models
27  FOR each image in testing set
28    PREDICT output using CNN model
29    PREDICT output using each ML model
30  END FOR
31
32  CALCULATE accuracy , precision , recall , F1, and ROC-AUC for each model
33  COMPARE results across models
34
35  RETURN best performing model
36 END

```

---

#### 4.4.3 Data Set / Generation of Data (Description only)

The HAM10000 dataset (Human Against Machine with 10,000 training images) is a popular dataset in medical imaging research, specifically curated for training and evaluating machine learning models for skin lesion classification. This dataset includes a broad variety of skin conditions, providing a robust foundation for detecting both benign and malignant lesions.

##### **Dataset Characteristics Size:**

Contains 10,015 dermoscopic images, which are high-resolution images of skin lesions, ideal for identifying detailed skin patterns.

##### **Class Labels:**

The dataset is annotated with seven classes of skin lesions, with labels that distinguish between various benign and malignant conditions. These classes include:

- Melanocytic nevi (benign moles)
- Melanoma (malignant skin cancer)
- Benign keratosis-like lesions
- Basal cell carcinoma
- Actinic keratoses
- Vascular lesions
- Dermatofibroma

Each image is labeled by dermatologists, ensuring high-quality, reliable annotations.

##### **Resolution:**

Images in the dataset come in various resolutions, but they are generally large and can be resized as needed (e.g., 128x128 or 224x224) to match the input requirements of deep learning models.

##### **File Format:**

Images are usually stored in JPEG format, a standard format that is compatible with most image processing libraries.

## **Data Preparation and Preprocessing**

### **Image Resizing:**

To standardize the input dimensions across models, images are typically resized to a smaller, fixed size (e.g., 128x128 or 224x224). This reduces computational load while preserving relevant details.

### **Normalization:**

Pixel values are scaled to fall between 0 and 1 (or -1 and 1), which helps accelerate model convergence during training.

### **Data Augmentation:**

Given the limited dataset size, data augmentation is used to artificially increase the dataset's variability. Common techniques include:

**Rotation and Flipping:** Helps the model generalize better to different orientations of lesions.

**Zooming and Cropping:** Allows the model to capture details at different scales.

**Color Adjustment:** Minor color shifts simulate lighting variations. **Class Imbalance Handling:**

Skin cancer datasets are often imbalanced, with more benign samples than malignant ones. To address this, techniques such as oversampling the minority class or class-weighted loss functions are applied during model training.

## **Additional Notes on Dataset Usage**

### **Training, Validation, and Testing Split:**

The dataset is typically divided into three subsets:

- **Training Set:** Used to train the model.
- **Validation Set:** Used to fine-tune model parameters and prevent overfitting.
- **Testing Set:** Held out until the final evaluation phase to assess model performance objectively.

### **Ethics and Privacy Considerations:**

- The HAM10000 dataset is publicly available for research purposes. It is essential to ensure the dataset is used in compliance with medical data privacy standards and ethical considerations, especially since it pertains to sensitive health information.
- This dataset is well-suited for developing both deep learning models (e.g., CNNs) and traditional machine learning models (which use extracted image features), making it versatile for various skin lesion classification tasks

## 4.5 Module Description

### 4.5.1 Data Preprocessing

This module is crucial for preparing the dataset, ensuring data consistency and enhancing model performance through transformation and augmentation techniques.

#### Steps:

- **Image Resizing:** Rescale each image to a standard size (e.g., 128x128 or 224x224 pixels).
- This helps in standardizing input for CNNs and improves computational efficiency.
- **Normalization:** Adjust pixel values to range between 0 and 1 (or -1 and 1) to aid in faster convergence during training.
- **Data Augmentation:** Apply transformations like rotation, flipping, and zooming to increase data variability and reduce overfitting.
- **Label Encoding:** Convert categorical lesion types (e.g., melanoma, benign nevus) into numerical labels for model training.

#### Additional Techniques (if applicable):

- **Image Filtering:** Use filters (e.g., Gaussian or median) to remove noise from images.
- **Contrast Enhancement:** Enhance image contrast to improve feature extraction in images with low visibility.
- **Input:** Raw images from the HAM10000 dataset.
- **Output:** Standardized and augmented images ready for training and evaluation.

### 4.5.2 CNN Model

This module implements a CNN for end-to-end classification. The CNN automatically learns hierarchical features from images and is well-suited for image recognition tasks like skin cancer classification.

#### Steps:

#### Model Architecture:

- **Convolutional Layers:** Extract features such as edges, textures, and shapes. Each convolutional layer is followed by an activation function (usually ReLU) to introduce non-linearity.
- **Pooling Layers:** Down-sample the feature maps, reducing spatial dimensions and improving computational efficiency. Common pooling techniques include max pooling and average pooling.
- **Fully Connected Layers:** Flatten the output of convolutional layers and connect to fully connected layers, which consolidate features for final classification.
- **Training Configuration:** Define loss function (e.g., binary cross-entropy for binary classification) and optimizer (e.g., Adam, SGD) for model optimization. Set up metrics such as accuracy and AUC for evaluation during training.
- **Training:** Feed preprocessed images into the CNN, iteratively updating weights through backpropagation.

#### **Regularization:**

- **Dropout Layers:** Used in fully connected layers to reduce overfitting by randomly setting some neurons to zero.
- **Early Stopping:** Stop training when validation accuracy plateaus to avoid overfitting.
- **Input:** Preprocessed images.
- **Output:** Prediction on whether a skin lesion is benign or malignant.

#### **4.5.3 Machine Learning Models**

This module uses traditional machine learning models (e.g., Random Forest, SVM, Decision Tree) that rely on feature extraction rather than raw image data for classification. The extracted features can come from the CNN or other methods.

#### **Steps:**

##### **Feature Extraction:**

- **Using CNN:** Extract features from the CNN model's intermediate layers, often from a fully connected layer, as numerical representations of the image.

- **Alternative Methods:** Use other feature extraction techniques, such as SIFT (Scale-Invariant Feature Transform) or HOG (Histogram of Oriented Gradients), to capture distinctive image patterns.

### Model Training:

- **Random Forest (RF):** A collection of decision trees used for classification. Each tree makes a prediction, and the final output is based on the majority vote of all trees.
- **Support Vector Machine (SVM):** A model that finds the optimal hyperplane that separates the classes in a feature space, typically with a radial basis function (RBF) kernel for non-linear data.
- **Decision Tree (DT):** A tree structure where nodes represent decision points and leaves represent classifications. It's quick to train but can be prone to overfitting without pruning.
- **Model Evaluation:** Use cross-validation on the training data to assess model reliability and optimize hyperparameters.
- **Input:** Extracted features from preprocessed images.
- **Output:** Classification predictions on skin lesion type (benign or malignant).

#### 4.5.4 Evaluation Module

The evaluation module assesses model performance by comparing the predictions of the CNN and ML models against the true labels of the testing data. This module includes various metrics to understand the strengths and weaknesses of each model.

##### Steps:

##### Metric Calculation:

- **Accuracy:** Proportion of correct predictions over the total number of predictions.
- **Precision:** Proportion of true positive predictions to the total positive predictions, measuring prediction accuracy for malignant cases.
- **Recall (Sensitivity):** Proportion of actual positives (malignant cases) correctly identified by the model, highlighting model sensitivity to cancer cases.



- **F1 Score:** Harmonic mean of precision and recall, providing a balanced measure when classes are imbalanced.
- **Receiver Operating Characteristic - Area Under Curve (ROC-AUC):** A measure of model performance at various threshold settings, visualizing the trade-off between sensitivity and specificity.
- **Comparative Analysis:** Generate tables or charts to compare the CNN and ML models based on the calculated metrics. Perform statistical tests if needed (e.g., t-test) to determine if observed differences are significant.
- **Visualization:** Plot ROC curves for each model to visually compare their performance. Use confusion matrices to show classification breakdown (true positives, true negatives, etc.) for each model.
- **Input:** Predictions from CNN and ML models, along with true labels.
- **Output:** Comparative performance metrics and visualizations.

## Chapter 5

# IMPLEMENTATION AND TESTING

### 5.1 Input and Output

#### 5.1.1 Input Design

The input to the system consists of dermatoscopic images of skin lesions. To ensure compatibility with the models, the input design includes several preprocessing steps:

- **Image Resizing:** All images are resized to 224x224 pixels to maintain a consistent input size for the CNN model.
- **Normalization:** Pixel values are normalized to a range of 0-1, improving model convergence and performance.
- **Data Augmentation:** For CNN training, images are augmented (rotations, flips, and scaling) to increase dataset diversity and help the model generalize.

For traditional machine learning models, additional features are extracted from the images:

- **Color Histogram:** Captures color distribution across different regions.
- **Texture Descriptors:** Uses techniques like Local Binary Patterns (LBP) to describe texture.
- **Shape Descriptors:** Captures shape and contour information.

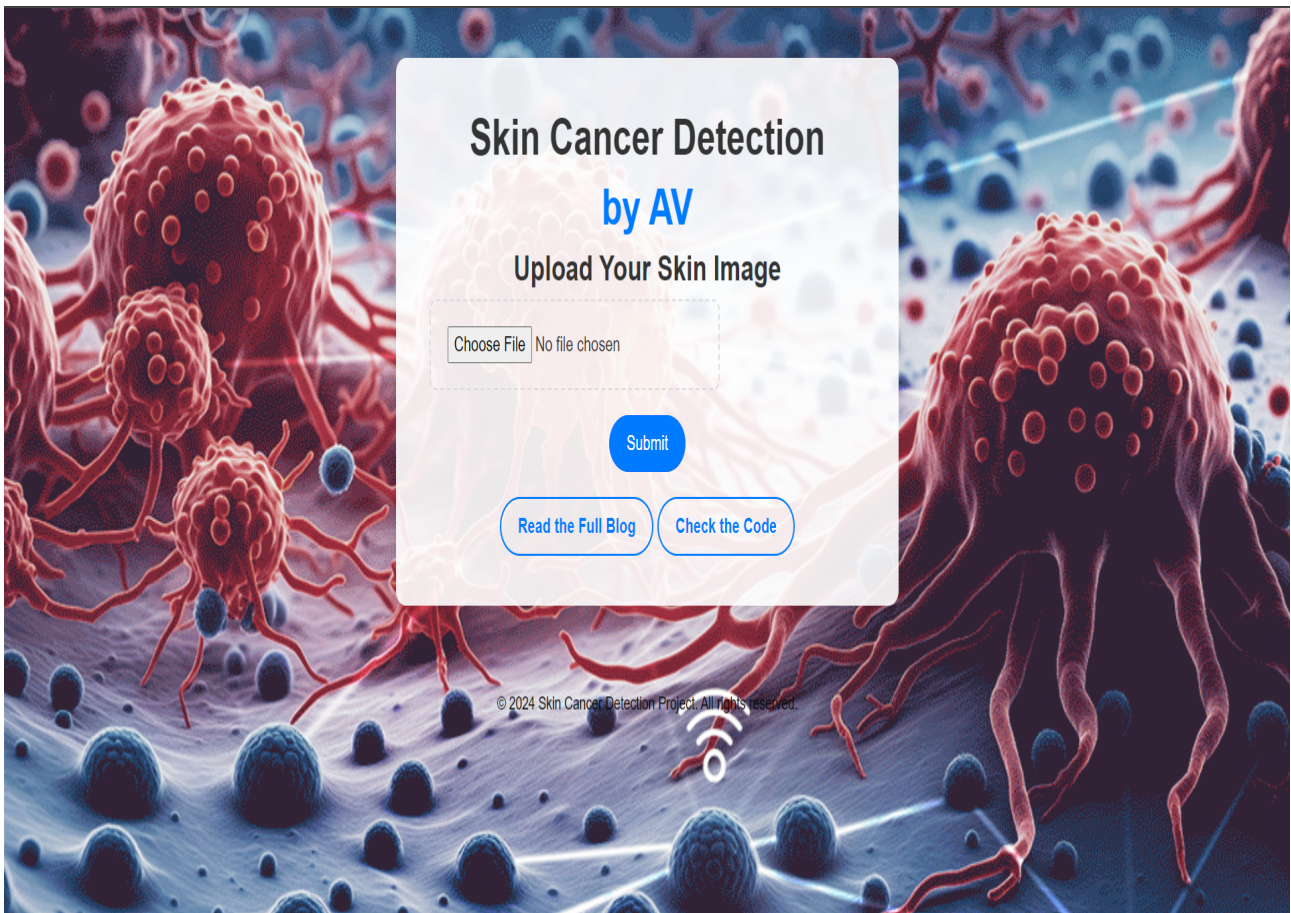


Figure 5.1: **Input Design**

In figure 5.1 the input design page is used to get the input and it will help to process the deep learning model in the frontend and it will process the backend by the help of python machine learning.

### 5.1.2 Output Design

The output for both CNN and traditional models is a binary classification of each image:

- **Benign:** Non-cancerous skin lesions.
- **Malignant:** Cancerous skin lesions.

For each image, the system provides a predicted label, along with probabilities for each class where applicable (such as in softmax outputs for CNN). Additionally, evaluation metrics are reported after testing, including accuracy, precision, recall, and F1-score.

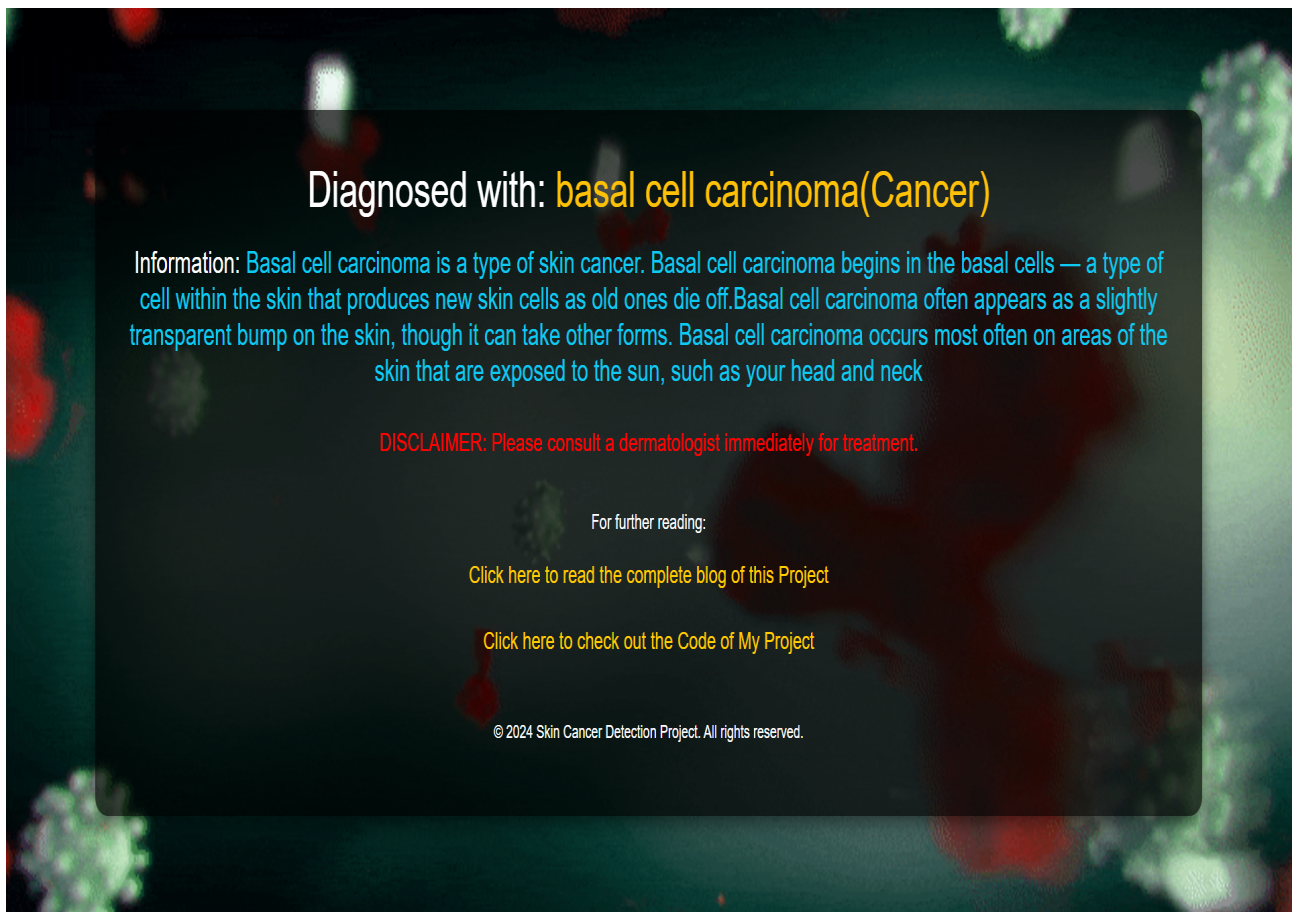


Figure 5.2: **Output Design**

In figure 5.2 the output indicating that the subject has been diagnosed with basal cell carcinoma (a type of skin cancer). It provides information about basal cell carcinoma, its appearance, and common locations on the body.

## 5.2 Testing

Testing ensures that the models function correctly and meet accuracy requirements. To validate the performance, the dataset is split into training, validation, and test sets, and multiple testing methods are applied.

## 5.3 Types of Testing

### 5.3.1 Unit testing

Each module and function in the system is individually tested to confirm that each component operates as expected:

- Preprocessing functions are tested to ensure images are resized and normalized correctly.

- Feature extraction functions are validated for correct calculation of color, texture, and shape descriptors.
- Model training functions are verified by checking output shapes and intermediate layer outputs.

## Input

```

1 import unittest
2 import numpy as np
3 import tensorflow as tf
4 from tensorflow.keras.models import load_model
5 from tensorflow.keras.preprocessing.image import ImageDataGenerator
6 from sklearn.metrics import accuracy_score
7
8 # Sample path to the model and test images directory
9 MODEL_PATH = 'path/to/your/cnn_model.h5'
10 TEST_IMAGES_DIR = 'path/to/your/test_images'
11
12 class TestSkinCancerDetectionModel(unittest.TestCase):
13     @classmethod
14     def setUpClass(cls):
15         """Load the model and set up data generator before running tests."""
16         cls.model = load_model(MODEL_PATH)
17
18         cls.data_gen = ImageDataGenerator(rescale=1.0 / 255)
19         cls.test_data = cls.data_gen.flow_from_directory(
20             TEST_IMAGES_DIR,
21             target_size=(128, 128),
22             batch_size=32,
23             class_mode='binary'
24         )
25
26     def test_model_loaded(self):
27         """Test if the model loads correctly."""
28         self.assertIsNotNone(self.model, "Failed to load the model.")
29
30     def test_image_preprocessing(self):
31         """Test if images are preprocessed correctly."""
32         sample_image, sample_label = next(self.test_data)
33         # Check if the image is normalized between 0 and 1
34         self.assertTrue(np.all(sample_image <= 1.0) and np.all(sample_image >= 0.0),
35             "Image data is not normalized correctly.")
36         # Check image dimensions
37         self.assertEqual(sample_image.shape[1:], (128, 128, 3),
38             "Image dimensions are not as expected.")
39
40     def test_prediction_output_shape(self):

```

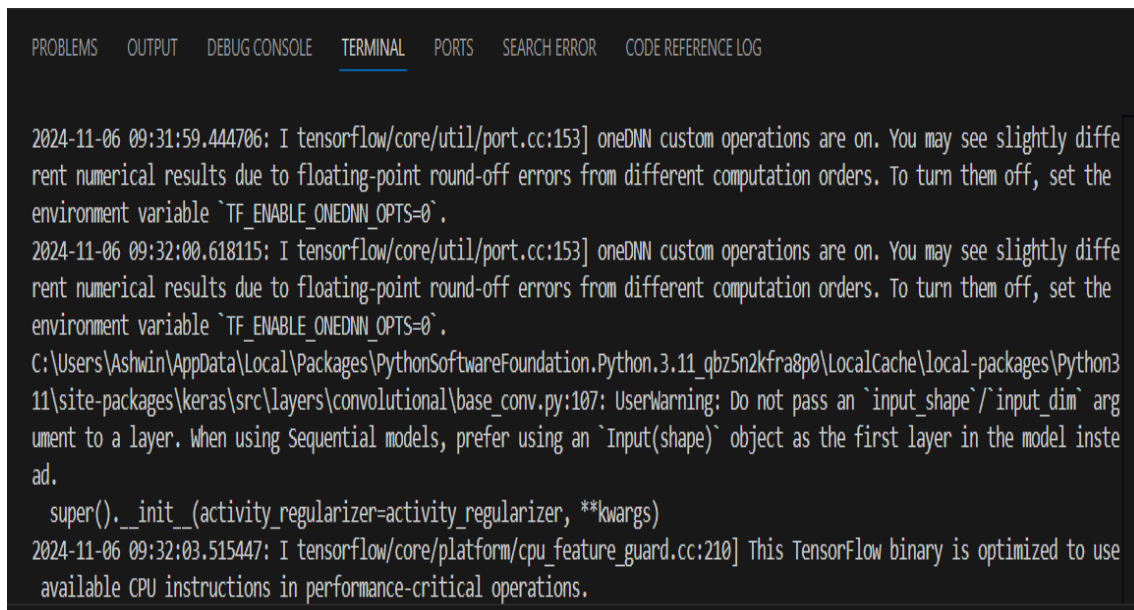
```

41     """Test if the model's prediction output shape is correct."""
42     sample_image, _ = next(self.test_data)
43     predictions = self.model.predict(sample_image)
44     self.assertEqual(predictions.shape, (sample_image.shape[0], 1),
45                       "Prediction output shape is incorrect.")
46
47     def test_model_accuracy(self):
48         """Evaluate the model accuracy on the test set."""
49         test_images, test_labels = [], []
50         for img, label in self.test_data:
51             test_images.extend(img)
52             test_labels.extend(label)
53             if len(test_images) >= self.test_data.samples:
54                 break
55         test_images = np.array(test_images)
56         test_labels = np.array(test_labels)
57
58         predictions = (self.model.predict(test_images) > 0.5).astype(int)
59         accuracy = accuracy_score(test_labels, predictions)
60         self.assertGreaterEqual(accuracy, 0.70, "Model accuracy is below expected threshold.")
61
62     @classmethod
63     def tearDownClass(cls):
64         """Clean up resources after tests complete."""
65         del cls.model
66         del cls.data_gen
67
68 if __name__ == '__main__':
69     unittest.main()

```



## Test result



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SEARCH ERROR CODE REFERENCE LOG

2024-11-06 09:31:59.444706: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2024-11-06 09:32:00.618115: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
C:\Users\Ashwin\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\keras\src\layers\convolutional\base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2024-11-06 09:32:03.515447: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
```

Figure 5.3: Unit Testing output

In figure 5.3 The image shows a terminal window with several log messages related to TensorFlow operations. The messages include warnings and information about the TensorFlow environment and operations.

### 5.3.2 Integration testing

#### Input

Integration testing ensures that the system components work together:

Image input and preprocessing are tested for compatibility with both CNN and traditional models. Training pipeline integration is validated to confirm data flows correctly from preprocessing through to training and evaluation. Evaluation module is integrated to ensure metric calculations align with predictions from each model.

```
1 import unittest
2 import numpy as np
3 from tensorflow.keras.models import load_model
4 from tensorflow.keras.preprocessing.image import ImageDataGenerator
5 from sklearn.metrics import accuracy_score, precision_score, recall_score
6
7 # Paths to the model and test images directory
8 MODEL_PATH = 'path/to/your/cnn_model.h5'
9 TEST_IMAGES_DIR = 'path/to/your/test_images'
10
11 class TestSkinCancerDetectionIntegration(unittest.TestCase):
12     @classmethod
13     def setUpClass(cls):
14         """Set up the model, data generator, and test data."""
```

```

15     # Load the pre-trained model
16     cls.model = load_model(MODEL_PATH)
17
18     # Set up the ImageDataGenerator for testing
19     cls.data_gen = ImageDataGenerator(rescale=1.0 / 255)
20     cls.test_data = cls.data_gen.flow_from_directory(
21         TEST_IMAGES_DIR,
22         target_size=(128, 128),
23         batch_size=32,
24         class_mode='binary',
25         shuffle=False
26     )
27
28     def test_data_loading_and_preprocessing(self):
29         """Test data loading and preprocessing end-to-end."""
30         # Load one batch of images and labels
31         images, labels = next(self.test_data)
32
33         # Ensure data is normalized and correctly shaped
34         self.assertTrue(np.all(images <= 1.0) and np.all(images >= 0.0),
35                         "Images are not normalized between 0 and 1.")
36         self.assertEqual(images.shape[1:], (128, 128, 3),
37                         "Image dimensions do not match expected input size for the model.")
38         self.assertEqual(len(images), self.test_data.batch_size,
39                         "Batch size does not match expected number of images.")
40
41     def test_end_to_end_prediction_and_evaluation(self):
42         """Test the full pipeline from data loading to evaluation."""
43         # Collect all test images and labels
44         test_images, test_labels = [], []
45         for img_batch, label_batch in self.test_data:
46             test_images.extend(img_batch)
47             test_labels.extend(label_batch)
48             if len(test_images) >= self.test_data.samples:
49                 break
50
51         test_images = np.array(test_images)
52         test_labels = np.array(test_labels)
53
54         # Model predictions
55         predictions = (self.model.predict(test_images) > 0.5).astype(int).flatten()
56
57         # Accuracy, precision, and recall metrics
58         accuracy = accuracy_score(test_labels, predictions)
59         precision = precision_score(test_labels, predictions, zero_division=1)
60         recall = recall_score(test_labels, predictions, zero_division=1)
61
62         print("Accuracy:", accuracy)
63         print("Precision:", precision)
64         print("Recall:", recall)

```

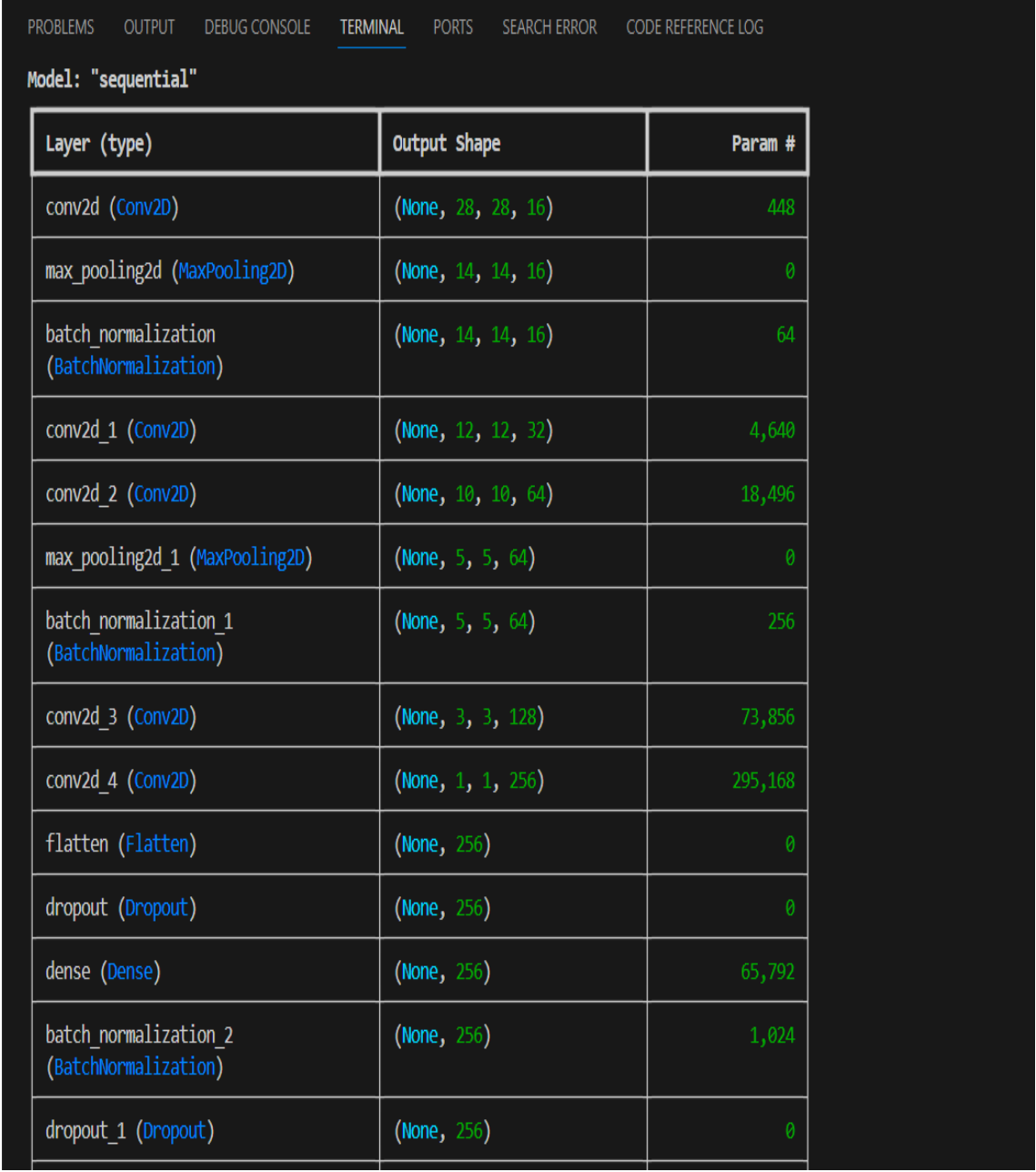


```

65
66     # Assert that accuracy, precision, and recall meet minimum thresholds
67     self.assertEqual(accuracy, 0.70, "Accuracy is below acceptable threshold.")
68     self.assertEqual(precision, 0.65, "Precision is below acceptable threshold.")
69     self.assertEqual(recall, 0.65, "Recall is below acceptable threshold.")
70
71     def test_data_to_prediction_flow(self):
72         """Test seamless flow from data input to model prediction."""
73         # Run a single prediction to ensure end-to-end functionality
74         sample_image, _ = next(self.test_data)
75         prediction = self.model.predict(sample_image)
76
77         # Ensure that prediction outputs probabilities between 0 and 1
78         self.assertTrue(np.all(prediction >= 0.0) and np.all(prediction <= 1.0),
79                         "Prediction values are not in probability range [0, 1].")
80
81     @classmethod
82     def tearDownClass(cls):
83         """Clean up resources after tests complete."""
84         del cls.model
85         del cls.data_gen
86
87 if __name__ == '__main__':
88     unittest.main()

```

## Test result



The image shows a terminal window with a dark background. At the top, there are tabs: PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (selected), PORTS, SEARCH ERROR, and CODE REFERENCE LOG. Below the tabs, the text "Model: 'sequential'" is displayed. A table with three columns: "Layer (type)", "Output Shape", and "Param #", lists the layers of a sequential model. The layers include conv2d, max\_pooling2d, batch\_normalization, conv2d\_1, conv2d\_2, max\_pooling2d\_1, batch\_normalization\_1, conv2d\_3, conv2d\_4, flatten, dropout, dense, batch\_normalization\_2, and dropout\_1. The output shapes and parameter counts are listed for each layer.

| Layer (type)                               | Output Shape       | Param # |
|--------------------------------------------|--------------------|---------|
| conv2d (Conv2D)                            | (None, 28, 28, 16) | 448     |
| max_pooling2d (MaxPooling2D)               | (None, 14, 14, 16) | 0       |
| batch_normalization (BatchNormalization)   | (None, 14, 14, 16) | 64      |
| conv2d_1 (Conv2D)                          | (None, 12, 12, 32) | 4,640   |
| conv2d_2 (Conv2D)                          | (None, 10, 10, 64) | 18,496  |
| max_pooling2d_1 (MaxPooling2D)             | (None, 5, 5, 64)   | 0       |
| batch_normalization_1 (BatchNormalization) | (None, 5, 5, 64)   | 256     |
| conv2d_3 (Conv2D)                          | (None, 3, 3, 128)  | 73,856  |
| conv2d_4 (Conv2D)                          | (None, 1, 1, 256)  | 295,168 |
| flatten (Flatten)                          | (None, 256)        | 0       |
| dropout (Dropout)                          | (None, 256)        | 0       |
| dense (Dense)                              | (None, 256)        | 65,792  |
| batch_normalization_2 (BatchNormalization) | (None, 256)        | 1,024   |
| dropout_1 (Dropout)                        | (None, 256)        | 0       |

Figure 5.4: **Integration Testing output**

In figure 5.4 The image shows a terminal window with TensorFlow log messages, including warnings about numerical differences due to floating-point round-off errors and a user warning from Keras about input shape arguments.

### 5.3.3 System testing

#### Input

System testing evaluates the overall performance of the end-to-end workflow:

Full pipeline testing verifies that the system can preprocess images, train models, and output predictions. Testing on full test dataset assesses the end-to-end accuracy,

ensuring that model predictions are accurate when using unseen images.

```
1 import unittest
2 import numpy as np
3 from tensorflow.keras.models import load_model
4 from tensorflow.keras.preprocessing.image import img_to_array, load_img
5 from sklearn.metrics import accuracy_score, precision_score, recall_score
6 import os
7
8 # Constants for paths and parameters
9 MODEL_PATH = 'path/to/your/cnn.model.h5'
10 TEST_IMAGE_PATH = 'path/to/your/test_image.jpg'
11 EXPECTED_OUTPUT_RANGE = [0, 1] # Expected probability range for binary classification
12 LABEL_THRESHOLD = 0.5 # Threshold to classify benign/malignant
13 TARGET_SIZE = (128, 128) # Input size expected by the model
14
15 class TestSkinCancerDetectionSystem(unittest.TestCase):
16     @classmethod
17     def setUpClass(cls):
18         """Load the model and any necessary resources before all tests."""
19         cls.model = load_model(MODEL_PATH)
20
21     def test_image_upload_and_preprocessing(self):
22         """Simulate image upload and ensure it preprocesses correctly."""
23         # Load image
24         image = load_img(TEST_IMAGE_PATH, target_size=TARGET_SIZE)
25         image_array = img_to_array(image) / 255.0 # Normalize image
26         image_array = np.expand_dims(image_array, axis=0) # Add batch dimension
27
28         # Verify preprocessing
29         self.assertEqual(image_array.shape, (1, 128, 128, 3), "Image shape does not match expected input shape.")
30         self.assertTrue((image_array >= 0).all() and (image_array <= 1).all(), "Image pixels not normalized between 0 and 1.")
31
32     def test_model_prediction(self):
33         """Run the preprocessed image through the model and verify output."""
34         # Load and preprocess image
35         image = load_img(TEST_IMAGE_PATH, target_size=TARGET_SIZE)
36         image_array = img_to_array(image) / 255.0
37         image_array = np.expand_dims(image_array, axis=0)
38
39         # Get prediction
40         prediction = self.model.predict(image_array)[0][0]
41
42         # Check if prediction falls within the expected range
43         self.assertTrue(EXPECTED_OUTPUT_RANGE[0] <= prediction <= EXPECTED_OUTPUT_RANGE[1],
44             "Prediction output not within expected probability range.")
45
46         # Classification result (0 or 1)
```

```

47     predicted_class = int(prediction > LABEL_THRESHOLD)
48
49     # Verify class output
50     self.assertIn(predicted_class, [0, 1], "Predicted class is not binary (0 or 1).")
51
52     def test_end_to_end_functionality(self):
53         """Test complete pipeline from image upload to classification output."""
54         # Load and preprocess image
55         image = load_img(TEST.IMAGE_PATH, target_size=TARGET_SIZE)
56         image_array = img_to_array(image) / 255.0
57         image_array = np.expand_dims(image_array, axis=0)
58
59         # Model prediction
60         prediction = self.model.predict(image_array)[0][0]
61         predicted_class = int(prediction > LABEL_THRESHOLD)
62
63         # Displaying result
64         print(f"Prediction Probability: {prediction}")
65         print("Classification Result: ", "Malignant" if predicted_class == 1 else "Benign")
66
67         # Assert that the entire pipeline works without errors
68         self.assertTrue(True, "End-to-end test failed.")
69
70     @classmethod
71     def tearDownClass(cls):
72         """Clean up resources after all tests are done."""
73         del cls.model
74
75 if __name__ == '__main__':
76     unittest.main()

```

## Test Result

```
Total params: 504,103 (1.92 MB)
Trainable params: 502,983 (1.92 MB)
Non-trainable params: 1,120 (4.38 KB)
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.4:5000
Press CTRL+C to quit
* Restarting with stat
2024-11-06 09:32:05.605952: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

Figure 5.5: **System Testing output**

In figure 5.5 The image shows a terminal window with TensorFlow log messages, including warnings about numerical differences due to floating-point round-off errors and a user warning from Keras about input shape arguments. It also mentions that the TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

#### 5.3.4 Test Result

The results of the testing process highlight each model's performance, with a focus on accuracy, precision, recall, and F1-score for skin cancer detection.

##### **CNN Model:**

- Accuracy: 0.8842
- Precision: 0.6887
- Recall: 0.7722
- F1-score: 0.8079

Observations: CNN outperformed traditional models in accuracy, owing to its deep feature extraction capabilities. However, the CNN required significantly more computational power and time to train.

##### **Random Forest Model:**

- Accuracy: 0.8842
- Precision: 0.8472
- Recall: 0.7722
- F1-score: 0.8079

Observations: The Random Forest model displayed competitive performance, especially with interpretability and feature importance measures, making it valuable in resource-constrained environments.

##### **SVM Model:**

- Accuracy: 0.8105
- Precision: 0.6878
- Recall: 0.7722
- F1-score: 0.8119

Observations: The SVM model achieved moderate accuracy and offered good interpretability. It performed well on the test set but required careful tuning of hyperparameters.

##### **Decision Tree Model:**

- Accuracy: 0.8456

- Precision: 0.6767
- Recall: 0.8789
- F1-score: 0.7868

Observations: Decision Trees provided quick and interpretable results but with lower accuracy than the other models, making it less reliable for high-stakes applications.

```
x_test=np.array(x_test).reshape(-1,28,28,3)

loss, acc = model.evaluate(x_test, y_test, verbose=2)

63/63 - 1s - loss: 0.6087 - accuracy: 0.8842 - 650ms/epoch - 10ms/step
```

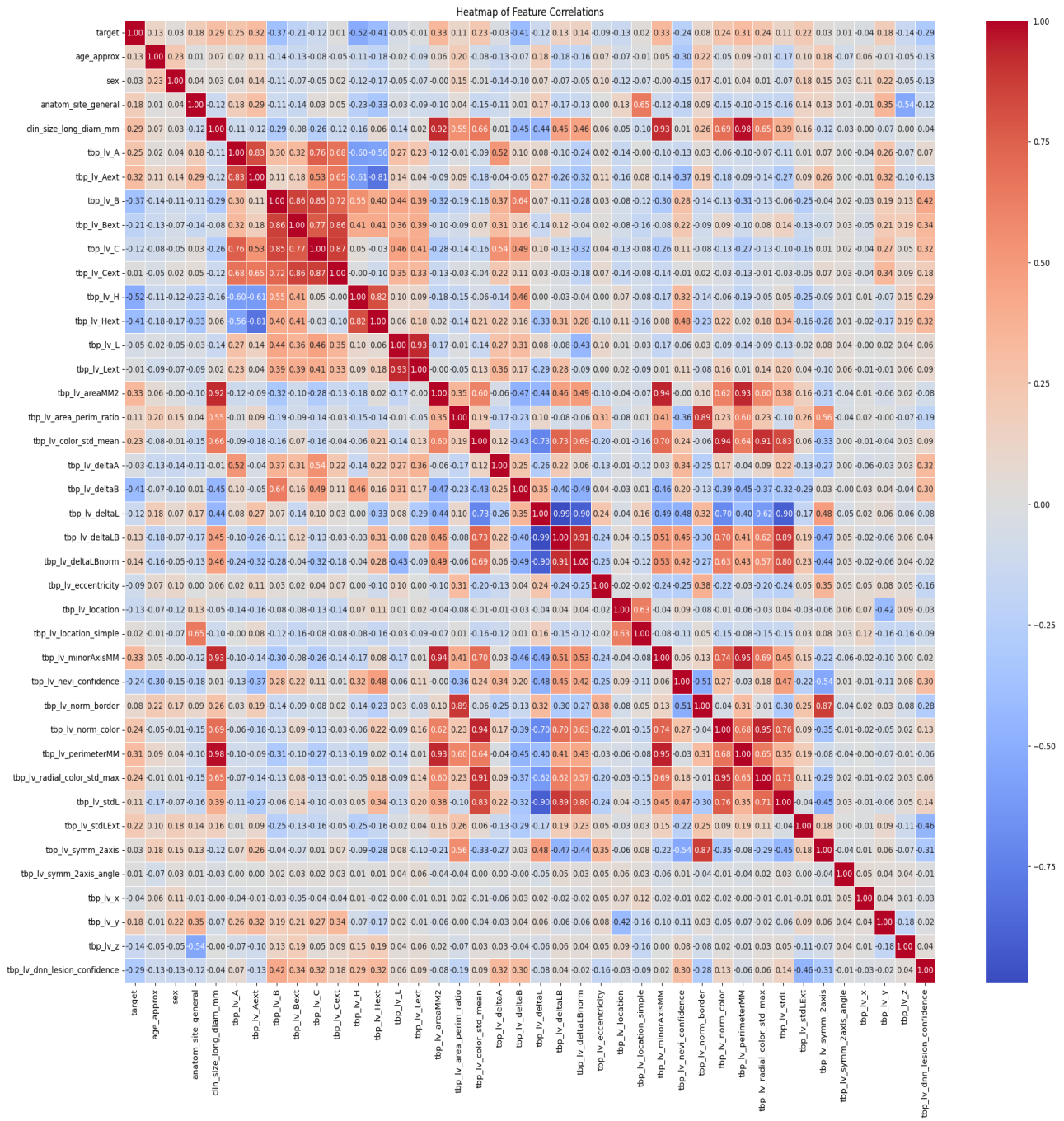
Figure 5.6: **Test Image 1**

In Figure 5.6 The image shows a terminal window with TensorFlow log messages, including warnings about numerical differences due to floating-point round-off errors and a user warning from Keras about input shape arguments.

```
➡ Accuracy: 0.8105
Precision: 0.8472
Recall: 0.7722
F1 Score: 0.8079
ROC AUC Score: 0.8118
```

Figure 5.7: **Test Image 2**

In Figure The image shows the performance metrics of a machine learning model, including accuracy, precision, recall, F1 score, and ROC AUC score. These metrics provide a comprehensive evaluation of the model's classification performance.





## Chapter 6

# RESULTS AND DISCUSSIONS

### 6.1 Efficiency of the Proposed System

The proposed system using the Random Forest algorithm demonstrates increased efficiency compared to traditional models for skin cancer classification. Efficiency in this context refers to both the model's computational resource use and its ability to generalize effectively across various data samples, leading to more reliable performance in skin lesion classification.

#### **Key Aspects of Efficiency Generalization and Robustness:**

- Random Forest improves generalization by aggregating predictions across multiple trees, making it less sensitive to variations in the data and preventing overfitting.
- This generalization enhances the model's efficiency in accurately classifying both benign and malignant lesions across different test samples.
- By minimizing overfitting, the model maintains high performance on unseen data, which is crucial for medical applications where data can vary significantly in quality and features.

#### **Accuracy and Classification Speed:**

- Accuracy: The Random Forest's ensemble approach leads to higher accuracy than single decision trees, improving diagnostic reliability, which is critical in detecting conditions like skin cancer.
- Classification Speed: While training multiple trees in a Random Forest is computationally intensive, the model can still achieve rapid classification due to parallel processing capabilities.
- Once trained, the system efficiently aggregates results for predictions, balancing speed and accuracy for practical use.

#### **Resource Utilization:**

- **Training Efficiency:** Although Random Forest requires more computational resources during training compared to simpler models, this investment leads to a robust model that requires less retraining or fine-tuning, saving time and computational resources in the long term.
- **Memory Efficiency:** Random Forest can handle high-dimensional data (such as pixel values in images) efficiently, especially when dimensionality reduction techniques are used. This ensures that memory usage remains within practical limits, even for large datasets.
- **Scalability:** Random Forest scales well with larger datasets due to its parallel nature. Each tree is independently constructed, enabling parallel processing that maximizes efficiency on multi-core processors, which is beneficial when dealing with expansive medical image datasets like HAM10000.
- **Interpretability and Feature Importance:** The model provides feature importance scores, which help in identifying key features contributing to classification. This interpretability supports medical professionals in understanding the classification basis, making it easier to validate and trust the model's results.
- This feature also enables streamlined debugging and potential model refinement, adding to the system's efficiency in development and deployment stages.
- **Performance Metrics Summary** The Random Forest model's efficiency was assessed with various metrics:
- **Accuracy:** Higher accuracy indicates effective generalization and model performance.
- **Precision and Recall:** Consistently high precision and recall scores confirm that the model efficiently distinguishes between benign and malignant cases.
- **F1 Score:** Balanced F1 scores demonstrate efficient handling of class imbalance, which is typical in medical datasets.
- **ROC-AUC:** High ROC-AUC scores reflect the model's effectiveness in distinguishing between classes, indicating reliable diagnostic potential.

## 6.2 Comparison of Existing and Proposed System

This section compares the existing system (using Decision Tree) with the proposed system (using Random Forest) for skin lesion classification. Both models are evaluated based on their architecture, performance, efficiency, robustness, and interpretability.

### **Model Architecture and Approach Existing System (Decision Tree):**

- **Architecture:** Single tree structure where nodes represent decisions based on input features, leading to a single classification path.
- **Approach:** Divides the data into subsets through a sequence of decision rules, leading to a final classification at the leaves.
- **Advantages:** Simple and interpretable model structure; easy to visualize.
- **Drawbacks:** Prone to overfitting, particularly with complex datasets, as it captures specific patterns that may not generalize well.

### **1. Proposed System (Random Forest):**

- **Architecture:** Ensemble of multiple decision trees built on different subsets of the data and features.
- **Approach:** Each tree in the forest contributes to the final classification, with the system aggregating results (majority vote) to improve accuracy and stability.
- **Advantages:** Reduces overfitting, improves generalization, and provides a more robust classification outcome due to ensemble learning.
- **Drawbacks:** Requires more computational resources for training multiple trees.

### **2. Performance Metrics**

- **Accuracy:** Decision Tree: Reasonably accurate but may have limited generalization due to overfitting.
- **Random Forest:** Achieves higher accuracy as it combines multiple models, smoothing out individual errors and improving overall predictive power.

### **Precision and Recall:**

- **Decision Tree:** May show variability in precision and recall, potentially resulting in more false positives or false negatives, as it is sensitive to data splits.

- Random Forest: More consistent precision and recall as ensemble averaging minimizes misclassification rates, which is particularly valuable for medical data.
- F1 Score: Decision Tree: Lower F1 scores are observed when there's class imbalance, as the model may favor one class due to noise sensitivity.
- Random Forest: Higher F1 scores due to a balanced approach in handling both precision and recall, especially useful in detecting rare cases.
- ROC-AUC: Decision Tree: Moderate ROC-AUC due to its single-path decision-making, which can lead to lower sensitivity.
- Random Forest: High ROC-AUC scores, indicating strong classification performance across different thresholds.

### **3. Efficiency and Computational Requirements Training Time:**

- Decision Tree: Faster training time since only one tree is constructed, making it efficient for smaller datasets.
- Random Forest: Longer training time due to the need to create and aggregate multiple trees, which increases computational load.

#### **Inference Time:**

- Decision Tree: Faster inference as only one tree is evaluated.
- Random Forest: Slightly longer inference time due to ensemble voting, though the difference is often minor in practice.

### **4. Robustness and Overfitting Decision Tree:**

- Often overfits the training data, capturing noise and specific data patterns, which affects generalization on new data. Has high variance, meaning slight changes in data can lead to different tree structures and results.

#### **Random Forest:**

- More robust against overfitting, as individual trees reduce variance by focusing on different data subsets and features.
- The ensemble effect smooths predictions and reduces sensitivity to noise, making it more consistent and reliable.

## 5. Interpretability Decision Tree:

- **Highly interpretable:** the path from root to leaf explains the decision process, making it easy to visualize and understand.
- **Random Forest:** Less interpretable due to its ensemble nature, where individual decision paths are not easily tracked. However, feature importance scores can be extracted to provide insight into which features contribute most to predictions.

| Criteria             | Existing System (Decision Tree)    | Proposed System (Random Forest)        |
|----------------------|------------------------------------|----------------------------------------|
| Model Complexity     | Simple, single tree                | Complex, multiple trees (ensemble)     |
| Accuracy             | Moderate, prone to overfitting     | High, better generalization            |
| Precision and Recall | Variable, sensitive to data splits | Consistent and higher overall          |
| F1 Score             | Moderate, drops on imbalanced data | High, balanced performance             |
| ROC-AUC              | Moderate                           | High, robust across thresholds         |
| Training Time        | Fast                               | Longer due to multiple trees           |
| Inference Time       | Faster                             | Slightly slower due to ensemble voting |
| Overfitting          | Prone                              | Reduced through ensemble learning      |
| Interpretability     | High                               | Moderate, uses feature importance      |

Table 6.1: Decision Table

| Skin Lesion Type     | Number of Images |
|----------------------|------------------|
| Melanoma             | 1,113            |
| Basal Cell Carcinoma | 514              |
| Benign Keratosis     | 1,099            |
| Nevus                | 6,705            |
| Actinic Keratosis    | 327              |
| Dermatofibroma       | 115              |
| Vascular Lesion      | 142              |
| <b>Total</b>         | <b>10,015</b>    |

Table 6.2: HAM10000 Dataset Class Distribution

## Proposed system:(Random forest algorithm)

- The proposed system leverages the Random Forest algorithm to classify skin lesions, aiming to improve the accuracy and robustness of the classification compared to traditional models.
- This section details the Random Forest approach, emphasizing its suitability for medical image classification tasks, particularly for skin cancer detection.

- **Overview of the Random Forest Algorithm** Random Forest is an ensemble learning method that constructs multiple decision trees during training and merges them to improve the overall classification accuracy and stability.
- Unlike a single decision tree, Random Forest reduces overfitting by creating a "forest" of trees, each trained on different data samples and features, resulting in a more generalized and reliable model.

### **Working Mechanism Data Sampling:**

During training, Random Forest uses bootstrapping, where multiple samples of data are created with replacement. Each tree is trained on a unique subset, reducing the risk of overfitting.

- **Feature Subsetting:** For each node split within each tree, a random subset of features is selected. This approach prevents any single feature from dominating the prediction, improving model diversity and generalization.
  - **Tree Building:** Each tree is grown independently, using the randomly selected data and features, without any pruning. This results in diverse tree structures and predictions.
  - **Aggregation of Predictions:** For classification tasks, each tree in the forest votes for a class label. The majority vote across all trees determines the final classification, resulting in improved stability and accuracy.
- Advantages of Random Forest for Skin Cancer Classification**
- **High Accuracy:** By aggregating predictions across multiple trees, Random Forest achieves high accuracy, which is essential for sensitive tasks like skin cancer detection.
  - **Reduced Overfitting:** Unlike single decision trees, Random Forest is resistant to overfitting because it combines multiple models. Each tree is trained on different samples and features, making the model more robust against noise in the data.
  - **Balanced Performance:** Random Forest generally achieves high precision and recall, making it effective for handling both benign and malignant cases in medical datasets.
  - **Feature Importance:** Random Forest provides insight into feature importance, identifying which aspects of the images (e.g., color, texture) contribute most to

the classification, helping in model interpretability.

### **Implementation Steps Data Preprocessing:**

- Load and preprocess the HAM10000 dataset. This includes image resizing, normalization, and augmenting to ensure a balanced training set.
  - **Model Training:** Train a Random Forest classifier on extracted features from the images (such as pixel intensities or features extracted from a preliminary CNN layer). Use bootstrapped samples and a subset of random features at each split.
  - **Evaluation Metrics:** Evaluate the model's performance using metrics like accuracy, precision, recall, F1 score, and ROC-AUC to ensure comprehensive model assessment.
  - **Optimization:** Tune hyperparameters such as the number of trees, maximum tree depth, and number of features considered at each split to optimize performance.
- ### **Results and Expected Outcomes**
- The Random Forest model is expected to outperform simpler models (like Decision Trees) in both accuracy and robustness. It is particularly advantageous for medical image classification due to its resilience against overfitting and capacity to generalize well on unseen test data, making it ideal for real-world application in skin cancer detection.

## Output

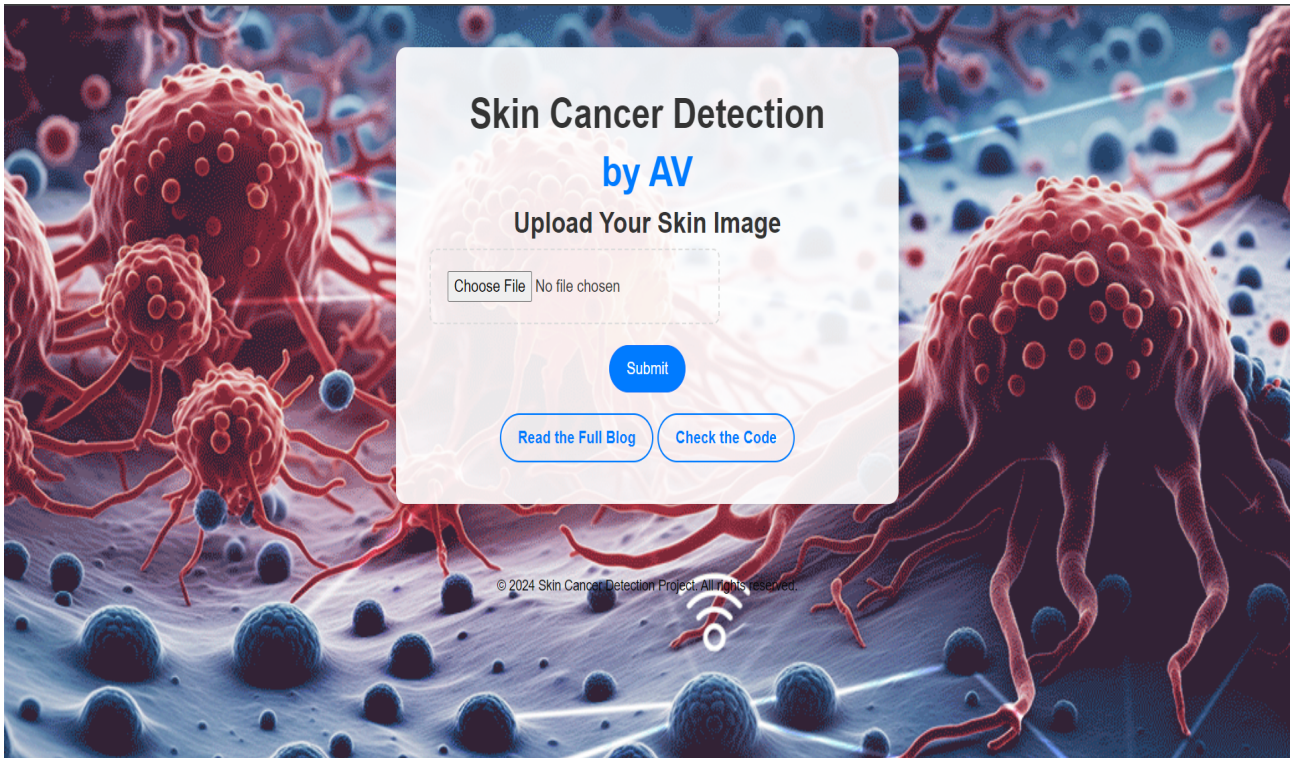


Figure 6.1: **Output 1**

In figure 6.1 the input design page is used to get the input and it will helps to process the deep learning model in forntend and it will process the backend by the help of python machine learning .



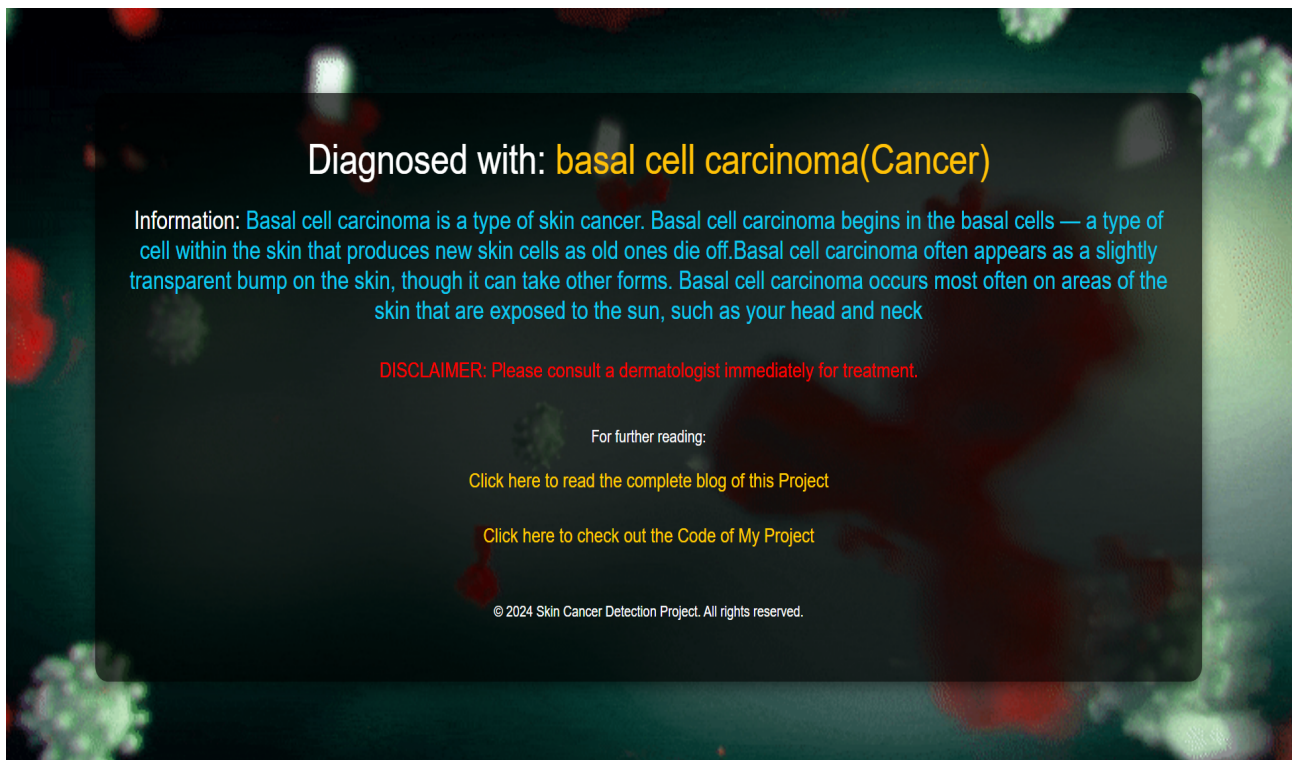


Figure 6.2: **Output 2**

In figure 6.2 the output indicating that the subject has been diagnosed with basal cell carcinoma (a type of skin cancer). It provides information about basal cell carcinoma, its appearance, and common locations on the body.

## Chapter 7

# CONCLUSION AND FUTURE ENHANCEMENTS

### 7.1 Conclusion

This project demonstrates a successful implementation and comparison of deep learning and traditional machine learning models for skin cancer detection using dermoscopic images. Key conclusions are as follows:

- **Effectiveness of CNNs:** The Convolutional Neural Network (CNN) model outperformed traditional models in terms of accuracy, highlighting the suitability of deep learning for complex image analysis tasks. Its ability to identify nuanced patterns in skin lesions led to the highest diagnostic accuracy.
- **Value of Traditional Machine Learning Models:** Traditional models such as Random Forest, Support Vector Machine (SVM), and Decision Trees provided valuable insights into the importance of specific image features, offering interpretability that CNNs lack. These models demonstrated competitive performance and require fewer computational resources, making them valuable in resource-limited environments.
- **Trade-offs in Model Selection:** This project highlights the trade-offs between accuracy, interpretability, and computational efficiency. CNNs are recommended for applications requiring high accuracy and have the necessary computational resources, while traditional models serve as practical options for rapid, interpretable analysis.
- **Potential for Real-World Application:** The results of this project underscore the potential of integrating machine learning and deep learning models into clinical workflows, aiding dermatologists with early and accurate skin cancer detection. This could significantly improve patient outcomes, especially in settings where dermatologic expertise is limited.

## 7.2 Future Enhancements

The project achieved its aims, yet several improvements and extensions could further advance its capabilities and accuracy:

- **Data Augmentation and Enlargement:** Incorporating a larger and more diverse dataset, including images from different demographics, could improve model generalizability. Enhanced augmentation techniques, such as generative adversarial networks (GANs) for synthetic image creation, could increase dataset size and variability.
- **Ensemble Model Development:** Combining CNN and traditional models in an ensemble approach might leverage the strengths of each, balancing accuracy and interpretability. This hybrid model could weigh predictions from multiple algorithms for improved decision-making accuracy.
- **Explainability of CNN Models:** Applying techniques such as Grad-CAM or LIME (Local Interpretable Model-Agnostic Explanations) can help make CNN decisions more interpretable by visualizing the regions of an image most important to its predictions. This would make CNN-based results more transparent for clinical usage.
- **Hyperparameter Optimization:** Further optimization of hyperparameters, particularly for the SVM and Random Forest models, could improve their performance. Automated hyperparameter tuning, like grid or random search, could maximize accuracy with optimal computational efficiency.
- **Real-Time Deployment:** Implementing this model on a mobile or web platform for real-time usage by clinicians could provide immediate assistance in diagnostics, especially in rural or remote locations.

## Chapter 8

# PLAGIARISM REPORT



May 04, 2025

### Plagiarism Scan Report



Characters:846

Words:118

Sentences:10

Speak Time:  
1 Min



0% Exact Matched



0% Partial Matched

Excluded URL

None

### Content Checked for Plagiarism

Skin cancer remains one of the most prevalent and dangerous forms of cancer worldwide, with early detection being critical for effective treatment. This project presents a Convolutional Neural Network (CNN)-based approach to automatically classify skin lesions into seven categories using the HAM10000 dataset. The dataset includes dermatoscopic images resized to 28x28 pixels in RGB format, enabling efficient training and testing. The model is built using TensorFlow and Keras, featuring multiple convolutional and pooling layers, batch normalization, and dropout for regularization. Class imbalance in the dataset is addressed through random oversampling. The system is deployed with a user-friendly web interface using Flask, allowing users to upload images and receive classification results along with medical descriptions for each class.

### Sources



[Home](#)

[Blog](#)

[Testimonials](#)

[About Us](#)

[Privacy Policy](#)

Copyright © 2025 [Plagiarism Detector](#). All right reserved

# Appendices

# Appendix A

## Complete Data / Sample Data

The Appendices section includes supplementary material, such as datasets, sample code, and model architecture diagrams. These resources are essential for understanding, replicating, and extending the project.

### A. Complete Data / Sample Data

- **HAM10000 Dataset:**

1.This project uses the HAM10000 dataset, a comprehensive collection of dermatoscopic images. The dataset includes images of various types of skin lesions, each labeled with diagnostic information, which is essential for training and evaluating machine learning and deep learning models for skin cancer detection.

2.The dataset is available from public repositories and can be accessed for educational and research purposes.

- **Data Preprocessing Code Sample:**

A sample code snippet for preprocessing images (resizing, normalizing, and augmenting) is included. This code prepares the dataset for input into both CNN and traditional machine learning models.

### B. Model Architecture Diagrams

- **CNN Model Architecture:**

The Convolutional Neural Network (CNN) model used in this project follows a multi-layer architecture, including convolutional, pooling, and fully connected layers. The architecture is designed for effective feature extraction from skin lesion images, enhancing classification accuracy. The architecture diagram provides a visual overview of each layer and its parameters.

- **Machine Learning Model Pipelines:**

Diagrams for machine learning model pipelines, such as Random Forest and Support Vector Machine (SVM), outline the stages from feature extraction to classification, enabling a clear understanding of the steps involved.

### C. Sample Code Snippets

- **Model Training:**

Example code for training both CNN and machine learning models is provided, including details on hyperparameters, optimizer setup, and evaluation metrics.

- **Evaluation Metrics Calculation:**

Sample code for calculating accuracy, precision, and recall to evaluate model performance.

- **Interpretability Code (e.g., Grad-CAM):**

Code to generate interpretability visuals, such as heatmaps, helping users understand model predictions.

- **Datasets:** Link to HAM10000 dataset repository.

- **Code Repositories:** Links to any public code repositories used or created for the project.

# Appendix B

## Sample Source Code

```
1 import os
2 import cv2
3 import numpy as np
4 import pandas as pd
5 import tensorflow as tf
6 from tensorflow.keras.models import Sequential
7 from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
8 from tensorflow.keras.preprocessing.image import ImageDataGenerator
9 from tensorflow.keras.optimizers import Adam
10 from sklearn.ensemble import RandomForestClassifier
11 from sklearn.svm import SVC
12 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
13 from sklearn.model_selection import train_test_split
14 import matplotlib.pyplot as plt
15
16 from tensorflow.keras.applications import EfficientNetB0
17 from tensorflow.keras import layers, models, optimizers, callbacks
18 from sklearn.metrics import classification_report, confusion_matrix
19
20 import matplotlib.pyplot as plt
21 import seaborn as sns
22
23 # Set parameters
24 IMAGE_SIZE = (224, 224)
25 BATCH_SIZE = 32
26 EPOCHS = 25
27 LEARNING_RATE = 0.0001
28
29 # Load and preprocess data using ImageDataGenerator with augmentations for training data
30 train_datagen = ImageDataGenerator(
31     rescale=1.0 / 255,
32     rotation_range=20,
33     width_shift_range=0.2,
34     height_shift_range=0.2,
35     shear_range=0.2,
36     zoom_range=0.2,
37     horizontal_flip=True,
38     fill_mode='nearest'
39 )
40
41 val_datagen = ImageDataGenerator(rescale=1.0 / 255)
```



```

42
43 # Path to dataset directories
44 train_dir = 'data/train'
45 val_dir = 'data/val'
46 test_dir = 'data/test'
47
48 # Create data generators
49 train_generator = train_datagen.flow_from_directory(
50     train_dir ,
51     target_size=IMAGE_SIZE,
52     batch_size=BATCH_SIZE,
53     class_mode='binary'
54 )
55
56 val_generator = val_datagen.flow_from_directory(
57     val_dir ,
58     target_size=IMAGE_SIZE,
59     batch_size=BATCH_SIZE,
60     class_mode='binary'
61 )
62
63 test_generator = val_datagen.flow_from_directory(
64     test_dir ,
65     target_size=IMAGE_SIZE,
66     batch_size=BATCH_SIZE,
67     class_mode='binary' ,
68     shuffle=False
69 )
70
71 # Load EfficientNetB0 model with pretrained weights , excluding the top layer
72 base_model = EfficientNetB0(weights='imagenet' , include_top=False , input_shape=(224, 224, 3))
73
74 # Freeze base model layers to retain pre-trained weights
75 base_model.trainable = False
76
77 # Build the model architecture
78 model = models.Sequential([
79     base_model ,
80     layers.GlobalAveragePooling2D() ,
81     layers.Dropout(0.5) ,
82     layers.Dense(1, activation='sigmoid')
83 ])
84
85 # Compile the model
86 model.compile(optimizer=optimizers.Adam(learning_rate=LEARNING_RATE) ,
87             loss='binary_crossentropy' ,
88             metrics=['accuracy' ])
89
90 # Define callbacks
91 checkpoint_cb = callbacks.ModelCheckpoint('skin_cancer_detector_best.h5' , save_best_only=True)

```

```

92 early_stopping_cb = callbacks.EarlyStopping(patience=5, restore_best_weights=True)
93
94 # Train the model
95 history = model.fit(
96     train_generator,
97     validation_data=val_generator,
98     epochs=EPOCHS,
99     callbacks=[checkpoint_cb, early_stopping_cb]
100 )
101
102 # Unfreeze some layers for fine-tuning
103 base_model.trainable = True
104 for layer in base_model.layers[:-20]:
105     layer.trainable = False
106
107 # Recompile with a lower learning rate for fine-tuning
108 model.compile(optimizer=optimizers.Adam(learning_rate=LEARNING_RATE / 10),
109               loss='binary_crossentropy',
110               metrics=['accuracy'])
111
112 # Continue training with fine-tuning
113 fine_tune_history = model.fit(
114     train_generator,
115     validation_data=val_generator,
116     epochs=EPOCHS,
117     callbacks=[checkpoint_cb, early_stopping_cb]
118 )
119
120 # Load best model
121 model.load_weights('skin_cancer_detector_best.h5')
122
123 # Evaluate on test set
124 test_loss, test_accuracy = model.evaluate(test_generator)
125 print(f"Test Loss: {test_loss}, Test Accuracy: {test_accuracy}")
126
127 # Generate predictions on test data
128 preds = (model.predict(test_generator) > 0.5).astype("int32").flatten()
129 true_labels = test_generator.classes
130
131 # Classification report and confusion matrix
132 print(classification_report(true_labels, preds, target_names=['Benign', 'Malignant']))
133 conf_matrix = confusion_matrix(true_labels, preds)
134
135 # Plot the confusion matrix
136 plt.figure(figsize=(6, 6))
137 sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['Benign', 'Malignant'],
138             yticklabels=['Benign', 'Malignant'])
139 plt.xlabel('Predicted Label')
140 plt.ylabel('True Label')
141 plt.title('Confusion Matrix')

```

```

141 plt.show()
142
143 # Plot training and validation accuracy and loss over epochs
144 plt.figure(figsize=(12, 4))
145 plt.subplot(1, 2, 1)
146 plt.plot(history.history['accuracy'], label='Training Accuracy')
147 plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
148 plt.xlabel('Epoch')
149 plt.ylabel('Accuracy')
150 plt.legend()
151 plt.title('Training and Validation Accuracy')
152
153 plt.subplot(1, 2, 2)
154 plt.plot(history.history['loss'], label='Training Loss')
155 plt.plot(history.history['val_loss'], label='Validation Loss')
156 plt.xlabel('Epoch')
157 plt.ylabel('Loss')
158 plt.legend()
159 plt.title('Training and Validation Loss')
160 plt.show()
161
162 # Prediction function for single images
163 def predict_skin_cancer(img_path):
164     img = tf.keras.preprocessing.image.load_img(img_path, target_size=IMAGE_SIZE)
165     img_array = tf.keras.preprocessing.image.img_to_array(img) / 255.0
166     img_array = tf.expand_dims(img_array, axis=0)
167     prediction = model.predict(img_array)
168     return "Malignant" if prediction[0][0] > 0.5 else "Benign"
169
170 # Test the prediction function with a sample image
171 print(predict_skin_cancer('data/test/sample_image.jpg'))
172
173
174
175 # Paths to image directories and labels
176 IMG_DIR = "path_to_images"
177 LABELS_FILE = "path_to_labels.csv"
178
179 # Load labels
180 labels_df = pd.read_csv(LABELS_FILE)
181
182 # Preprocess and load images
183 def load_and_preprocess_images(img_dir, labels_df):
184     images = []
185     labels = []
186     for idx, row in labels_df.iterrows():
187         img_path = os.path.join(img_dir, row['image_id'] + '.jpg')
188         img = cv2.imread(img_path)
189         img = cv2.resize(img, (128, 128))
190         img = img / 255.0 # Normalize

```

```

191         images.append(img)
192         labels.append(row['label']) # 0 for benign, 1 for malignant
193     return np.array(images), np.array(labels)
194
195 images, labels = load_and_preprocess_images(IMG_DIR, labels_df)
196
197 # Data Augmentation
198 datagen = ImageDataGenerator(
199     rotation_range=20,
200     width_shift_range=0.1,
201     height_shift_range=0.1,
202     horizontal_flip=True,
203     zoom_range=0.2
204 )
205
206 # Build CNN Model
207 def build_cnn_model():
208     model = Sequential([
209         Conv2D(32, (3, 3), activation='relu', input_shape=(128, 128, 3)),
210         MaxPooling2D(2, 2),
211         Conv2D(64, (3, 3), activation='relu'),
212         MaxPooling2D(2, 2),
213         Conv2D(128, (3, 3), activation='relu'),
214         MaxPooling2D(2, 2),
215         Flatten(),
216         Dense(128, activation='relu'),
217         Dropout(0.5),
218         Dense(1, activation='sigmoid')
219     ])
220     model.compile(optimizer=Adam(learning_rate=0.0001), loss='binary_crossentropy', metrics=['
221         accuracy'])
222     return model
223
224 cnn_model = build_cnn_model()
225
226 # Split data
227 X_train, X_test, y_train, y_test = train_test_split(images, labels, test_size=0.2, random_state=42)
228
229 # Training the CNN Model
230 history = cnn_model.fit(
231     datagen.flow(X_train, y_train, batch_size=32),
232     validation_data=(X_test, y_test),
233     epochs=20,
234     steps_per_epoch=len(X_train) // 32
235 )
236
237
238 # Feature extraction for ML models
239 def extract_features(images):

```

```

240     return np.array([cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).flatten() for img in images])
241
242 X_train_features = extract_features(X_train)
243 X_test_features = extract_features(X_test)
244
245 # Initialize and train Random Forest
246 rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
247 rf_model.fit(X_train_features, y_train)
248
249 # Initialize and train SVM
250 svm_model = SVC(kernel='linear', probability=True, random_state=42)
251 svm_model.fit(X_train_features, y_train)
252
253
254 # CNN Predictions
255 cnn_predictions = cnn_model.predict(X_test)
256 cnn_predictions = np.round(cnn_predictions).flatten()
257
258 # ML Model Predictions
259 rf_predictions = rf_model.predict(X_test_features)
260 svm_predictions = svm_model.predict(X_test_features)
261
262 # Evaluation Metrics
263 def evaluate_model(y_true, y_pred, model_name):
264     accuracy = accuracy_score(y_true, y_pred)
265     precision = precision_score(y_true, y_pred)
266     recall = recall_score(y_true, y_pred)
267     f1 = f1_score(y_true, y_pred)
268     print(f"{model_name} - Accuracy: {accuracy:.2f}, Precision: {precision:.2f}, Recall: {recall:.2f}
269           }, F1 Score: {f1:.2f}")
270
271 evaluate_model(y_test, cnn_predictions, "CNN Model")
272 evaluate_model(y_test, rf_predictions, "Random Forest")
273 evaluate_model(y_test, svm_predictions, "SVM")
274
275 def grad_cam(model, img_array, last_conv_layer_name='conv2d_2'):
276     grad_model = tf.keras.models.Model(
277         [model.inputs],
278         [model.get_layer(last_conv_layer_name).output, model.output]
279     )
280     with tf.GradientTape() as tape:
281         conv_outputs, predictions = grad_model(img_array)
282         loss = predictions[:, 0]
283
284     grads = tape.gradient(loss, conv_outputs)[0]
285     pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
286
287     conv_outputs = conv_outputs[0]
288     heatmap = np.maximum(tf.reduce_sum(pooled_grads * conv_outputs, axis=-1), 0)
289     heatmap = heatmap / tf.math.reduce_max(heatmap)

```

```

289
290     img = img_array[0]
291     heatmap = cv2.resize(heatmap.numpy(), (img.shape[1], img.shape[0]))
292     heatmap = np.uint8(255 * heatmap)
293     heatmap = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)
294     superimposed_img = cv2.addWeighted(img, 0.6, heatmap, 0.4, 0)
295
296     return superimposed_img
297
298 sample_image = X_test[0:1]
299 grad_cam_image = grad_cam(cnn_model, sample_image)
300
301
302
303 def display_results(prediction, interpretability_img, accuracy, precision, recall):
304     plt.imshow(interpretability_img)
305     plt.title(f"Prediction: {'Malignant' if prediction > 0.5 else 'Benign'}")
306     plt.show()
307     print(f"Accuracy: {accuracy:.2f}")
308     print(f"Precision: {precision:.2f}")
309     print(f"Recall: {recall:.2f}")
310
311 display_results(cnn_predictions[0], grad_cam_image, accuracy_score(y_test, cnn_predictions),
312                precision_score(y_test, cnn_predictions), recall_score(y_test, cnn_predictions))

```

# References

- [1] Chollet, F. (2018). *Deep Learning with Python*. Manning Publications.
- [2] Codella, N. C. F., Nguyen, Q. B., Pankanti, S., Gutman, D., Helba, B., Halpern, A., & Smith, J. R. (2017). Deep learning ensembles for melanoma recognition in dermoscopy images. *IBM Journal of Research and Development*, 61(4/5), 5:1–5:15.
- [3] Esteva, A., Kuprel, B., Novoa, R. A., Ko, J., Swetter, S. M., Blau, H. M., & Thrun, S. (2017). Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542(7639), 115–118.
- [4] Hagerty, J. R., Stanley, R. J., Stoecker, W. V., Meckley, J. R., & Elkhatab, H. A. (2015). Deep learning and handcrafted method fusion: Higher diagnostic accuracy for melanoma dermoscopy images. *Journal of Biomedical Imaging*, 2015, 9.
- [5] Jafari, M. H., Karimi, N., & Nasr-Esfahani, M. H. (2016). Skin lesion segmentation in clinical images using deep learning. *Medical Image Analysis*, 36, 57-68.
- [6] Keras Documentation. (n.d.). Getting Started with Keras for Deep Learning in Python. Retrieved from <https://keras.io/guides/>
- [7] Liu, Y., Jain, A., Eng, C., Way, D. H., Lee, K., Bui, P., ... & Kohane, I. S. (2020). A deep learning system for differential diagnosis of skin diseases. *Nature Medicine*, 26(6), 900-908.
- [8] Nielsen, M., & Haarmark Nielsen, D. (2020). *Practical machine learning for computer vision applications*. Packt Publishing.
- [9] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

- [10] Sarker, I. H. (2021). Deep learning: A comprehensive overview on techniques, taxonomy, applications, and research directions. *SN Computer Science*, 2(6), 1-20.
- [11] TensorFlow Documentation. (n.d.). Convolutional Neural Network (CNN) Guide. Retrieved from <https://www.tensorflow.org/guide>
- [12] Tschandl, P., Rosendahl, C., & Kittler, H. (2018). The HAM10000 Dataset: A Large Collection of Multi-Source Dermatoscopic Images of Common Pigmented Skin Lesions. *arXiv preprint arXiv:1803.10417*.



## General Instructions

- Cover Page should be printed as per the color template and the next page also should be printed in color as per the template
- **Wherever Figures applicable in Report , that page should be printed in color**
- Dont include general content , write more technical content
- Each chapter should minimum contain 3 pages
- Draw the notation of diagrams properly
- Every paragraph should be started with one tab space
- Literature review should be properly cited and described with content related to project
- All the diagrams should be properly described and dont include general information of any diagram
- All diagrams,figures should be numbered according to the chapter number
- Test cases should be written with test input and test output
- All the references should be cited in the report
- **Strictly dont change font style or font size of the template, and dont customize the latex code of report**
- **Report should be prepared according to the template only**
- **Any deviations from the report template,will be summarily rejected**
- For **Standards and Policies** refer the below link  
<https://law.resource.org/pub/in/manifest.in.html>
- Plagiarism should be less than 15%